home.js

```
const express = require('express');
const { upload, storageService } = require('./storage');
const path = require('path');
const fs = require('fs');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const cookieParser = require('cookie-parser');
require('dotenv').config();
const { query, getClient } = require('./db/config');
const app = express();
const PORT = process.env.PORT || 3005;
// Middleware
app.use(express.static('public'));
app.use(express.json());
app.use(cookieParser());
// ==============================================
// PATIENT API ROUTES - Copied from Patient.js
// ==============================================

// ==============================================
// IN-MEMORY USER DATABASE (Replace with PostgreSQL later)
// ==============================================

// Track active sessions
let activeSessions = new Map();

// Create upload directories if they don't exist
const uploadDirs = [
    './uploads',
    './uploads/reports',
    './uploads/scans',
    './uploads/temp'
];

uploadDirs.forEach(dir => {
    if (!fs.existsSync(dir)) {
        fs.mkdirSync(dir, { recursive: true });
        console.log(`Created directory: ${dir}`);
    }
});
// ==============================================
// JWT HELPER FUNCTIONS
// ==============================================
const generateToken = (user) => {
    return jwt.sign(
```

```
        {
            id: user.id || user.user_id,
            username: user.username,
            role: user.role
        },
        process.env.JWT_SECRET || 'fallback_secret',
        { expiresIn: process.env.JWT_EXPIRE || '7d' }
    );
};

const verifyToken = (token) => {
    try {
        return jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret');
    } catch (error) {
        return null;
    }
};

// ============================================
// AUTH MIDDLEWARE
// ============================================
const authenticate = (req, res, next) => {
    let token = req.cookies.token;

    if (!token && req.headers.authorization) {
        const authHeader = req.headers.authorization;
        if (authHeader.startsWith('Bearer ')) {
            token = authHeader.substring(7);
        }
    }

    if (!token) {
        return res.status(401).json({ success: false, message: 'Authentication required' });
    }

    const decoded = verifyToken(token);
    if (!decoded) {
        return res.status(401).json({ success: false, message: 'Invalid or expired token' });
    }

    if (!activeSessions.has(decoded.id)) {
        return res.status(401).json({ success: false, message: 'Session expired' });
    }

    req.user = decoded;
    next();
};
```

```
const authorize = (...roles) => {
    return (req, res, next) => {
        if (!req.user) {
            return res.status(401).json({ success: false, message: 'Authentication required' });
        }
        if (!roles.includes(req.user.role)) {
            return res.status(403).json({ success: false, message: 'Access denied' });
        }
        next();
    };
};

const requireAuth = (role) => {
    return (req, res, next) => {
        const token = req.cookies.token;
        if (!token) return res.redirect('/signin');

        const decoded = verifyToken(token);
        if (!decoded || !activeSessions.has(decoded.id)) {
            return res.redirect('/signin');
        }

        if (role && decoded.role !== role) {
            return res.status(403).send(`
                <h1>403 - Access Denied</h1>
                <p>You don't have permission to access this page.</p>
                <a href="/">Go to Home</a>
            `);
        }

        req.user = decoded;
        next();
    };
};

// ==============================================
// AUTHENTICATION ROUTES
// ==============================================

// Register
app.post('/api/auth/register', async (req, res) => {
  console.log('📝 Registration attempt:', req.body);
  const client = await getClient();
  try {
    await client.query('BEGIN');

    const { name, email, password, gender,blood_type, phone, address, dob, ...rest } =
req.body;
```

```
    const role = 'patient';
    const username = email.split('@')[0]; // Create username from email

    console.log('Extracted data:', { name, email, username, gender,blood_type, phone,
address, dob });

    // Check if user exists
    const existing = await client.query(
      'SELECT * FROM users WHERE username = $1 OR email = $2',
      [username, email]
    );

    if (existing.rows.length > 0) {
      await client.query('ROLLBACK');
      return res.status(400).json({ success: false, message: 'User already exists' });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    // Insert user
    const userResult = await client.query(
      `INSERT INTO users (username, email, password_hash, role)
       VALUES ($1, $2, $3, $4) RETURNING user_id, username, email, role`,
      [username, email, hashedPassword, role]
    );

    const user = userResult.rows[0];
    console.log('✅ User created:', user);

    // Insert patient profile (without emergency contact fields since they're not in the form)
    const date = new Date();
    const year = date.getFullYear();
    const month = String(date.getMonth() + 1).padStart(2, '0');
    const day = String(date.getDate()).padStart(2, '0');
    const random = Math.floor(Math.random() * 10000).toString().padStart(4, '0');
    const patient_uuid = `PT-${year}${month}${day}-${random}`
    await client.query(
      `INSERT INTO patients (
         user_id, patient_uuid, full_name, email, phone, address, date_of_birth, gender,
blood_type
    ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)`,
      [
        user.user_id,patient_uuid, name, email, phone, address, dob, gender, blood_type
      ]
    );
    console.log('✅ Patient profile created');
```

```
    await client.query('COMMIT');
    console.log('✅ Registration successful');

    const token = generateToken({ id: user.user_id, username, role: user.role });
    activeSessions.set(user.user_id, { token, loginTime: new Date().toISOString() });

    res.cookie('token', token, { httpOnly: true, maxAge: 7 * 24 * 60 * 60 * 1000 });

    res.json({
      success: true,
      token,
      user: { id: user.user_id, username, email, role: user.role }
    });
  } catch (error) {
    await client.query('ROLLBACK');
    console.error('❌ Registration error:', error);
    res.status(500).json({ success: false, message: 'Server error: ' + error.message });
  } finally {
    client.release();
  }
});


// ==============================================
// FILE UPLOAD ROUTES
// ==============================================

// Upload lab report with file
// Upload lab report with file
app.post('/api/lab/upload-report', authenticate, authorize('lab'), upload.single('report'), async
(req, res) => {
    const client = await getClient();
    try {
        await client.query('BEGIN');

        if (!req.file) {
            return res.status(400).json({ success: false, message: 'No file uploaded' });
        }

        const { patientId, doctorId, testType, priority, findings, sendTo } = req.body;

        // Get lab technician ID
        const labTechResult = await client.query(
            'SELECT lab_tech_id FROM lab_technicians WHERE user_id = $1',
            [req.user.id]
        );

        if (labTechResult.rows.length === 0) {
```

```
        return res.status(404).json({ success: false, message: 'Lab technician not found' });
    }

    const labTechId = labTechResult.rows[0].lab_tech_id;

    // Get patient_id from patient_uuid (friendly ID like PT-20260306-3907)
    const patientResult = await client.query(
        'SELECT patient_id FROM patients WHERE patient_uuid = $1',
        [patientId]
    );

    if (patientResult.rows.length === 0) {
        return res.status(404).json({ success: false, message: 'Patient not found with ID: ' +
patientId });
    }

    const actualPatientId = patientResult.rows[0].patient_id;

    // Get doctor_id from doctor_uuid (friendly ID like DR-2024-0567) if provided
    let actualDoctorId = null;
    if (doctorId && doctorId.trim() !== '') {
        const doctorResult = await client.query(
            'SELECT doctor_id FROM doctors WHERE doctor_uuid = $1',
            [doctorId]
        );

        if (doctorResult.rows.length > 0) {
            actualDoctorId = doctorResult.rows[0].doctor_id;
        }
    }

    // Upload file info
    const fileData = await storageService.uploadFile(req.file, 'reports');

    // Generate report UUID
    const reportUUID = 'REP-' + Date.now() + '-' + Math.random().toString(36).substr(2, 6);

    // Get today's date for test_date
    const today = new Date().toISOString().split('T')[0];

    // Insert into database
    const result = await client.query(
        `INSERT INTO lab_reports (
            report_uuid, patient_id, doctor_id, lab_tech_id,
            test_type, test_date, findings, file_url,
            status, priority, shared_with
        ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11)
        RETURNING report_id, report_uuid`,
```

```
            [
                reportUUID,
                actualPatientId,
                actualDoctorId,
                labTechId,
                testType,
                today,
                findings || 'No findings',
                fileData.url,
                'completed',
                priority || 'normal',
                sendTo  // This is the key - using the value from the form directly
            ]
        );

        await client.query('COMMIT');

        res.json({
            success: true,
            message: 'Report uploaded successfully',
            reportId: result.rows[0].report_uuid,
            sharedWith: sendTo // Return the value for debugging
        });

    } catch (error) {
        await client.query('ROLLBACK');
        console.error('Error uploading report:', error);
        res.status(500).json({ success: false, message: error.message });
    } finally {
        client.release();
    }
});

// Serve uploaded files
app.get('/uploads/:folder/:file', (req, res) => {
    const filePath = path.join(__dirname, 'uploads', req.params.folder, req.params.file);
    if (fs.existsSync(filePath)) {
        res.sendFile(filePath);
    } else {
        res.status(404).json({ error: 'File not found' });
    }
});

// Get report file for download
app.get('/api/reports/:reportId/download', authenticate, async (req, res) => {
    try {
        const { reportId } = req.params;
```

```
        const result = await query(
            `SELECT file_url, report_uuid, test_type FROM lab_reports WHERE report_uuid =
$1`,
            [reportId]
        );

        if (result.rows.length === 0) {
            return res.status(404).json({ error: 'Report not found' });
        }

        const report = result.rows[0];

        // Extract filename from URL
        const fileName = report.file_url.split('/').pop();
        const filePath = path.join(__dirname, 'uploads/reports', fileName);

        if (fs.existsSync(filePath)) {
            res.download(filePath, `${report.test_type}_${report.report_uuid}.pdf`);
        } else {
            res.status(404).json({ error: 'File not found on server' });
        }
    } catch (error) {
        console.error('Error downloading report:', error);
        res.status(500).json({ error: error.message });
    }
});

// Get patient list for lab technician
app.get('/api/lab/patients', authenticate, authorize('lab'), async (req, res) => {
    try {
        const result = await query(
            `SELECT patient_id, patient_uuid, full_name FROM patients ORDER BY full_name`
        );
        res.json(result.rows);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// Get doctors list for lab technician
app.get('/api/lab/doctors', authenticate, authorize('lab'), async (req, res) => {
    try {
        const result = await query(
            `SELECT doctor_id, doctor_uuid, full_name, specialization FROM doctors WHERE
status = 'Available' ORDER BY full_name`
        );
        res.json(result.rows);
    } catch (error) {
```

```
        res.status(500).json({ error: error.message });
    }
});

// Add these routes to home.js - near your other API routes

// ==============================================
// HOSPITAL MANAGEMENT API ROUTES
// ==============================================

// Get hospital data for dashboard
app.get('/api/hospital/data', authenticate, async (req, res) => {
  try {
    // Get the hospital_id from the admin's hospital
    const adminResult = await query(
      'SELECT hospital_id FROM hospital_admins WHERE user_id = $1',
      [req.user.id]
    );

    const hospitalId = adminResult.rows[0]?.hospital_id;

    if (!hospitalId) {
      return res.status(404).json({ error: 'Hospital not found' });
    }

    const hospitalResult = await query('SELECT * FROM hospitals WHERE hospital_id = $1',
[hospitalId]);
    const doctorsResult = await query('SELECT * FROM doctors WHERE hospital_id = $1',
[hospitalId]);
    const medicinesResult = await query('SELECT * FROM medicines WHERE hospital_id =
$1', [hospitalId]);
    const labsResult = await query('SELECT * FROM lab_technicians WHERE hospital_id =
$1', [hospitalId]);

    res.json({
      hospitalName: hospitalResult.rows[0]?.name || 'City General Hospital',
      hospitalId: hospitalResult.rows[0]?.hospital_uuid || 'HOS-12345',
      doctors: doctorsResult.rows,
      medicines: medicinesResult.rows,
      labs: labsResult.rows,
      specialities: [...new Set(doctorsResult.rows.map(d => d.specialization).filter(Boolean))]
    });
  } catch (error) {
    console.error('Error fetching hospital data:', error);
    res.status(500).json({ error: error.message });
  }
});
```

```
// ==============================================
// REVIEWS API ROUTES - Complete Review System
// ==============================================

// Get all approved reviews with sorting and filtering
app.get('/api/reviews', async (req, res) => {
  try {
    const {
      sort = 'recent',
      rating,
      search,
      hospital,
      doctor,
      page = 1,
      limit = 10
    } = req.query;

    let whereClause = 'WHERE is_approved = true';
    const params = [];
    let paramIndex = 1;

    // Filter by rating
    if (rating) {
      if (rating === 'positive') {
        whereClause += ` AND rating >= 4`;
      } else if (rating === 'negative') {
        whereClause += ` AND rating <= 2`;
      } else if (rating === 'neutral') {
        whereClause += ` AND rating = 3`;
      } else if (!isNaN(parseInt(rating))) {
        whereClause += ` AND rating = $${paramIndex}`;
        params.push(parseInt(rating));
        paramIndex++;
      }
    }

    // Search in content or title
    if (search) {
      whereClause += ` AND (content ILIKE $${paramIndex} OR title ILIKE $${paramIndex}
OR reviewer_name ILIKE $${paramIndex})`;
      params.push(`%${search}%`);
      paramIndex++;
    }

    // Filter by hospital
    if (hospital) {
      whereClause += ` AND hospital_name ILIKE $${paramIndex}`;
      params.push(`%${hospital}%`);
```

```
  paramIndex++;
}

// Filter by doctor
if (doctor) {
  whereClause += ` AND doctor_name ILIKE $${paramIndex}`;
  params.push(`%${doctor}%`);
  paramIndex++;
}

// Determine sort order
let orderBy = 'created_at DESC';
if (sort === 'highest') {
  orderBy = 'rating DESC, created_at DESC';
} else if (sort === 'lowest') {
  orderBy = 'rating ASC, created_at DESC';
} else if (sort === 'helpful') {
  orderBy = 'helpful_count DESC, created_at DESC';
}

// Get total count for pagination
const countResult = await query(
  `SELECT COUNT(*) FROM reviews ${whereClause}`,
  params
);
const totalCount = parseInt(countResult.rows[0].count);

// Add pagination
const offset = (page - 1) * limit;
const paginatedParams = [...params, limit, offset];

const result = await query(
  `SELECT
    review_id,
    reviewer_name,
    rating,
    title,
    content,
    hospital_name,
    doctor_name,
    is_verified,
    helpful_count,
    created_at,
    updated_at,
    CASE
      WHEN created_at >= CURRENT_DATE THEN 'Today'
      WHEN created_at >= CURRENT_DATE - INTERVAL '1 day' THEN 'Yesterday'
      ELSE TO_CHAR(created_at, 'Mon DD, YYYY')
```

```
      END as formatted_date
      FROM reviews
      ${whereClause}
      ORDER BY ${orderBy}
      LIMIT $${paramIndex} OFFSET $${paramIndex + 1}`,
    paginatedParams
  );

  // Get rating statistics
  const statsResult = await query(
    `SELECT
      COUNT(*) as total_reviews,
      COALESCE(AVG(rating), 0) as average_rating,
      COUNT(CASE WHEN rating = 5 THEN 1 END) as five_star,
      COUNT(CASE WHEN rating = 4 THEN 1 END) as four_star,
      COUNT(CASE WHEN rating = 3 THEN 1 END) as three_star,
      COUNT(CASE WHEN rating = 2 THEN 1 END) as two_star,
      COUNT(CASE WHEN rating = 1 THEN 1 END) as one_star
      FROM reviews
      WHERE is_approved = true`
  );

  res.json({
    success: true,
    reviews: result.rows,
    stats: statsResult.rows[0],
    pagination: {
      currentPage: parseInt(page),
      totalPages: Math.ceil(totalCount / limit),
      totalCount,
      limit: parseInt(limit)
    }
  });

 } catch (error) {
  console.error('Error fetching reviews:', error);
  res.status(500).json({ success: false, message: error.message });
 }
});

// Submit a new review (authenticated users only)
app.post('/api/reviews', authenticate, async (req, res) => {
 const client = await getClient();
 try {
  await client.query('BEGIN');

  const { rating, title, content, hospital_name, doctor_name } = req.body;
```

```
    // Validate input
    if (!rating || rating < 1 || rating > 5) {
      return res.status(400).json({ success: false, message: 'Rating must be between 1 and 5'
});
    }

    if (!content || content.trim().length < 10) {
      return res.status(400).json({ success: false, message: 'Review content must be at least
10 characters' });
    }

    // Get patient info if user is a patient
    let patientId = null;
    let reviewerName = req.user.username;
    let reviewerEmail = null;

    if (req.user.role === 'patient') {
      const patientResult = await client.query(
        'SELECT patient_id, full_name, email FROM patients WHERE user_id = $1',
        [req.user.id]
      );
      if (patientResult.rows.length > 0) {
        patientId = patientResult.rows[0].patient_id;
        reviewerName = patientResult.rows[0].full_name || reviewerName;
        reviewerEmail = patientResult.rows[0].email;
      }
    }

    // Insert review
    const result = await client.query(
      `INSERT INTO reviews (
        user_id, patient_id, reviewer_name, reviewer_email,
        rating, title, content, hospital_name, doctor_name,
        is_verified, is_approved, helpful_count
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, 0)
      RETURNING *`,
      [
        req.user.id,
        patientId,
        reviewerName,
        reviewerEmail,
        rating,
        title || null,
        content,
        hospital_name || null,
        doctor_name || null,
        req.user.role === 'patient', // Verified if they're a patient
        true, // Auto-approve for now (change to false if you want moderation)
```

```
      ]
    );

    await client.query('COMMIT');

    res.status(201).json({
      success: true,
      message: 'Review submitted successfully',
      review: result.rows[0]
    });

  } catch (error) {
    await client.query('ROLLBACK');
    console.error('Error submitting review:', error);
    res.status(500).json({ success: false, message: error.message });
  } finally {
    client.release();
  }
});

// Mark a review as helpful
app.post('/api/reviews/:id/helpful', async (req, res) => {
  try {
    const { id } = req.params;

    const result = await query(
      `UPDATE reviews
       SET helpful_count = helpful_count + 1
       WHERE review_id = $1::uuid
       RETURNING helpful_count`,
      [id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ success: false, message: 'Review not found' });
    }

    res.json({
      success: true,
      helpful_count: result.rows[0].helpful_count
    });

  } catch (error) {
    console.error('Error updating helpful count:', error);
    res.status(500).json({ success: false, message: error.message });
  }
});
```

```
// Get unique hospitals and doctors for filters
app.get('/api/reviews/filters', async (req, res) => {
  try {
    const hospitalsResult = await query(
      `SELECT DISTINCT hospital_name
       FROM reviews
       WHERE hospital_name IS NOT NULL AND is_approved = true
       ORDER BY hospital_name
       LIMIT 50`
    );

    const doctorsResult = await query(
      `SELECT DISTINCT doctor_name
       FROM reviews
       WHERE doctor_name IS NOT NULL AND is_approved = true
       ORDER BY doctor_name
       LIMIT 50`
    );

    res.json({
      success: true,
      hospitals: hospitalsResult.rows.map(r => r.hospital_name),
      doctors: doctorsResult.rows.map(r => r.doctor_name)
    });

  } catch (error) {
    console.error('Error fetching filters:', error);
    res.status(500).json({ success: false, message: error.message });
  }
});

//UPDATE LEAVE
app.post('/api/doctors/:doctorId/leave', authenticate, authorize('admin'), async (req, res) => {
  const client = await getClient();
  try {
    await client.query('BEGIN');

    const { doctorId } = req.params;
    const { from, to, reason } = req.body;

    // Validate UUID format
    const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
    if (!uuidRegex.test(doctorId)) {
      await client.query('ROLLBACK');
      return res.status(400).json({ success: false, message: 'Invalid doctor ID format' });
    }

    // Get hospital_id from the admin
```

```
    const adminResult = await client.query(
      `SELECT hospital_id FROM hospital_admins WHERE user_id = $1`,
      [req.user.id]
    );

    const hospitalId = adminResult.rows[0]?.hospital_id;

    if (!hospitalId) {
      await client.query('ROLLBACK');
      return res.status(404).json({ success: false, message: 'Hospital not found for this admin'
});
    }

    // Verify the doctor belongs to this admin's hospital
    const doctorCheck = await client.query(
      `SELECT * FROM doctors WHERE doctor_id = $1::uuid AND hospital_id = $2::uuid`,
      [doctorId, hospitalId]
    );

    if (doctorCheck.rows.length === 0) {
      await client.query('ROLLBACK');
      return res.status(404).json({ success: false, message: 'Doctor not found in your hospital'
});
    }

    // Insert leave record into doctor_leave table
    const leaveResult = await client.query(
      `INSERT INTO doctor_leave (doctor_id, from_date, to_date, reason, status)
       VALUES ($1::uuid, $2::date, $3::date, $4, 'Approved')
       RETURNING *`,
      [doctorId, from, to, reason]
    );

    // Update doctor status to 'On Leave'
    await client.query(
      `UPDATE doctors SET status = 'On Leave' WHERE doctor_id = $1::uuid`,
      [doctorId]
    );

    await client.query('COMMIT');

    res.json({
      success: true,
      message: 'Leave updated successfully',
      data: leaveResult.rows[0]
    });
  } catch (error) {
    await client.query('ROLLBACK');
```

```
    console.error('Error updating leave:', error);
    res.status(500).json({ success: false, message: error.message });
  } finally {
    client.release();
  }
});

// Get all leave records for a specific doctor
app.get('/api/doctors/:doctorId/leave', authenticate, authorize('admin', 'doctor'), async (req,
res) => {
  try {
    const { doctorId } = req.params;

    // Validate UUID format
    const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
    if (!uuidRegex.test(doctorId)) {
      return res.status(400).json({ success: false, message: 'Invalid doctor ID format' });
    }

    // Check authorization
    if (req.user.role === 'doctor') {
      // Verify the doctor is viewing their own leave records
      const doctorCheck = await query(
        `SELECT doctor_id FROM doctors WHERE user_id = $1::uuid`,
        [req.user.id]
      );

      if (doctorCheck.rows.length === 0 || doctorCheck.rows[0].doctor_id !== doctorId) {
        return res.status(403).json({ success: false, message: 'You can only view your own
leave records' });
      }
    }

    const result = await query(
      `SELECT * FROM doctor_leave
       WHERE doctor_id = $1::uuid
       ORDER BY created_at DESC`,
      [doctorId]
    );

    res.json({ success: true, data: result.rows });
  } catch (error) {
    console.error('Error fetching leave records:', error);
    res.status(500).json({ success: false, message: error.message });
  }
});

// Get all doctors on leave (for admin dashboard)
```

```
app.get('/api/doctors/on-leave', authenticate, authorize('admin'), async (req, res) => {
  try {
    // Get hospital_id from the admin
    const adminResult = await query(
      `SELECT hospital_id FROM hospital_admins WHERE user_id = $1`,
      [req.user.id]
    );

    const hospitalId = adminResult.rows[0]?.hospital_id;

    if (!hospitalId) {
      return res.status(404).json({ success: false, message: 'Hospital not found' });
    }

    const result = await query(
      `SELECT d.*, dl.from_date, dl.to_date, dl.reason, dl.leave_id
       FROM doctors d
       JOIN doctor_leave dl ON d.doctor_id = dl.doctor_id
       WHERE d.hospital_id = $1::uuid
         AND d.status = 'On Leave'
         AND dl.status = 'Approved'
         AND dl.from_date <= CURRENT_DATE
         AND dl.to_date >= CURRENT_DATE
       ORDER BY d.full_name`,
      [hospitalId]
    );

    res.json({ success: true, data: result.rows });
  } catch (error) {
    console.error('Error fetching doctors on leave:', error);
    res.status(500).json({ success: false, message: error.message });
  }
});

// Remove/end leave (when doctor returns)
// REPLACE the existing DELETE /api/doctors/:doctorId/leave route in home.js with this fixed
version.
// Bug fixed: PostgreSQL does not support ORDER BY / LIMIT directly in UPDATE
statements.
// Solution: use a subquery to find the leave_id first, then update by that id.

app.delete('/api/doctors/:doctorId/leave', authenticate, authorize('admin'), async (req, res) =>
{
  const client = await getClient();
  try {
    await client.query('BEGIN');

    const { doctorId } = req.params;
```

```
    // Validate UUID format
    const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
    if (!uuidRegex.test(doctorId)) {
      await client.query('ROLLBACK');
      return res.status(400).json({ success: false, message: 'Invalid doctor ID format' });
    }

    // Get hospital_id from the admin
    const adminResult = await client.query(
      `SELECT hospital_id FROM hospital_admins WHERE user_id = $1`,
      [req.user.id]
    );

    const hospitalId = adminResult.rows[0]?.hospital_id;
    if (!hospitalId) {
      await client.query('ROLLBACK');
      return res.status(404).json({ success: false, message: 'Hospital not found' });
    }

    // Update doctor status back to 'Available'
    const doctorUpdate = await client.query(
      `UPDATE doctors SET status = 'Available'
       WHERE doctor_id = $1::uuid AND hospital_id = $2::uuid
       RETURNING *`,
      [doctorId, hospitalId]
    );

    if (doctorUpdate.rows.length === 0) {
      await client.query('ROLLBACK');
      return res.status(404).json({ success: false, message: 'Doctor not found in your hospital'
});
    }

    // FIX: PostgreSQL does not support ORDER BY / LIMIT in UPDATE.
    // Find the most recent active leave_id first, then update it.
    await client.query(
      `UPDATE doctor_leave
       SET status = 'Completed'
       WHERE leave_id = (
         SELECT leave_id
         FROM doctor_leave
         WHERE doctor_id = $1::uuid
           AND status = 'Approved'
           AND to_date >= CURRENT_DATE
         ORDER BY created_at DESC
         LIMIT 1
       )`,
```

```
      [doctorId]
    );

    await client.query('COMMIT');

    res.json({
      success: true,
      message: 'Leave removed, doctor is now available',
      data: doctorUpdate.rows[0]
    });
  } catch (error) {
    await client.query('ROLLBACK');
    console.error('Error removing leave:', error);
    res.status(500).json({ success: false, message: error.message });
  } finally {
    client.release();
  }
});

// ... existing leave delete route above ...

app.delete('/api/doctors/:doctorId', authenticate, authorize('admin'), async (req, res) => {
  const client = await getClient();
  try {
    await client.query('BEGIN');

    const { doctorId } = req.params;

    const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
    if (!uuidRegex.test(doctorId)) {
      await client.query('ROLLBACK');
      return res.status(400).json({ success: false, message: 'Invalid doctor ID format' });
    }

    const adminResult = await client.query(
      'SELECT hospital_id FROM hospital_admins WHERE user_id = $1',
      [req.user.id]
    );
    const hospitalId = adminResult.rows[0]?.hospital_id;
    if (!hospitalId) {
      await client.query('ROLLBACK');
      return res.status(404).json({ success: false, message: 'Hospital not found' });
    }

    const doctorCheck = await client.query(
      'SELECT user_id FROM doctors WHERE doctor_id = $1::uuid AND hospital_id =
$2::uuid',
      [doctorId, hospitalId]
```

```
    );
    if (doctorCheck.rows.length === 0) {
      await client.query('ROLLBACK');
      return res.status(404).json({ success: false, message: 'Doctor not found in your hospital'
});
    }

    const doctorUserId = doctorCheck.rows[0].user_id;

    await client.query(`UPDATE appointments SET status = 'cancelled' WHERE doctor_id =
$1::uuid AND appointment_date >= CURRENT_DATE`, [doctorId]);
    await client.query(`UPDATE doctors SET status = 'Inactive' WHERE doctor_id = $1::uuid`,
[doctorId]);
    await client.query(`UPDATE users SET is_active = false WHERE user_id = $1::uuid`,
[doctorUserId]);


    await client.query('COMMIT');
    res.json({ success: true, message: 'Doctor removed successfully' });

  } catch (error) {
    await client.query('ROLLBACK');
    console.error('Error deleting doctor:', error);
    res.status(500).json({ success: false, message: error.message });
  } finally {
    client.release();
  }
});

// Add new doctor
app.post('/api/hospital/add/doctor', authenticate, authorize('admin'), async (req, res) => {
  const client = await getClient();
  try {
    await client.query('BEGIN');

    const { name, speciality, phone, email, designation, dateOfBirth, education,
            medicalCouncil, registrationNumber, idType, idNumber, appointmentDate } =
req.body;

    // Get hospital_id from the admin
    const adminResult = await client.query(
      'SELECT hospital_id FROM hospital_admins WHERE user_id = $1',
      [req.user.id]
    );

    const hospitalId = adminResult.rows[0]?.hospital_id;

    if (!hospitalId) {
```

```
    await client.query('ROLLBACK');
    return res.status(404).json({ success: false, error: 'Hospital not found' });
  }

  // Create username from email
  const username = email ? email.split('@')[0] : `doctor_${Date.now()}`;

  // Create temporary password (you might want to generate a random one)
  const salt = await bcrypt.genSalt(10);
  const tempPassword = await bcrypt.hash('Welcome@123', salt);

  // Create user account for doctor
  const userResult = await client.query(
    `INSERT INTO users (username, email, password_hash, role, hospital_id)
     VALUES ($1, $2, $3, $4, $5) RETURNING user_id`,
    [username, email || `${username}@bondhealth.com`, tempPassword, 'doctor', hospitalId]
  );

  const userId = userResult.rows[0].user_id;

  // Generate doctor_uuid
  const doctorUuid = `DR-${Date.now()}`;

  // Add doctor to doctors table
  const doctorResult = await client.query(
    `INSERT INTO doctors (
      user_id, hospital_id, doctor_uuid, full_name, specialization,
      phone, email, designation, qualification, status
     ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)
     RETURNING *`,
    [
      userId, hospitalId, doctorUuid, name, speciality,
      phone, email, designation || 'Consultant', education || 'MBBS', 'Available'
    ]
  );

  await client.query('COMMIT');

  res.json({
    success: true,
    message: 'Doctor registered successfully',
    data: doctorResult.rows[0]
  });
} catch (error) {
  await client.query('ROLLBACK');
  console.error('Error adding doctor:', error);
  res.status(500).json({ success: false, error: error.message });
} finally {
```

```
    client.release();
  }
});

// Add new medicine
app.post('/api/hospital/add/medicine', authenticate, async (req, res) => {
  try {
    const adminResult = await query(
      'SELECT hospital_id FROM hospital_admins WHERE user_id = $1',
      [req.user.id]
    );

    const hospitalId = adminResult.rows[0]?.hospital_id;

    if (!hospitalId) {
      return res.status(404).json({ success: false, error: 'Hospital not found' });
    }

    const { name, quantity, price, expiry } = req.body;

    const result = await query(
      `INSERT INTO medicines (hospital_id, name, quantity, price, expiry_date)
       VALUES ($1, $2, $3, $4, $5)
       RETURNING *`,
      [hospitalId, name, quantity, price, expiry]
    );

    res.json({ success: true, data: result.rows[0] });
  } catch (error) {
    console.error('Error adding medicine:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Add new lab technician
app.post('/api/hospital/add/lab', authenticate, async (req, res) => {
  try {
    const adminResult = await query(
      'SELECT hospital_id FROM hospital_admins WHERE user_id = $1',
      [req.user.id]
    );

    const hospitalId = adminResult.rows[0]?.hospital_id;

    if (!hospitalId) {
      return res.status(404).json({ success: false, error: 'Hospital not found' });
    }
```

```
    const labData = req.body;

    // Create user for lab technician
    const salt = await bcrypt.genSalt(10);
    const tempPassword = await bcrypt.hash('Welcome@123', salt);

    const userResult = await query(
      `INSERT INTO users (username, email, password_hash, role, hospital_id)
       VALUES ($1, $2, $3, $4, $5) RETURNING user_id`,
      [labData.name.replace(/\s+/g, '').toLowerCase(), labData.email ||
`${labData.name}@bondhealth.com`, tempPassword, 'lab', hospitalId]
    );

    const result = await query(
      `INSERT INTO lab_technicians (user_id, hospital_id, full_name, employee_id, phone,
email)
       VALUES ($1, $2, $3, $4, $5, $6)
       RETURNING *`,
      [userResult.rows[0].user_id, hospitalId, labData.name, labData.licenseNumber,
labData.phone, labData.email]
    );

    res.json({ success: true, data: result.rows[0] });
  } catch (error) {
    console.error('Error adding lab:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Add this route in home.js - place it near your other dashboard routes
app.get('/register-doctor', requireAuth('admin'), async (req, res) => {
    try {
        const Hospital = require('./Hospital.js');
        const hospitalHTML = Hospital.getAddDoctorHTML();
        res.setHeader('Content-Type', 'text/html');
        res.send(hospitalHTML);
    } catch (err) {
        console.error('Error loading doctor registration:', err);
        res.status(500).send(`
            <h1>500 - Error</h1>
            <p>Could not load doctor registration page.</p>
            <a href="/admin-dashboard">← Back to Dashboard</a>
        `);
    }
});

// Add these routes to home.js (near your other routes)
```

```
// Add Speciality page
app.get('/add-speciality', requireAuth('admin'), async (req, res) => {
    try {
        const Hospital = require('./Hospital.js');
        const html = Hospital.getAddSpecialityHTML();
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading add speciality:', err);
        res.status(500).send('Error loading page');
    }
});

// Add Doctor page (what you currently have)
app.get('/add-doctor', requireAuth('admin'), async (req, res) => {
    try {
        const Hospital = require('./Hospital.js');
        const html = Hospital.getAddDoctorHTML();
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading add doctor:', err);
        res.status(500).send('Error loading page');
    }
});

// Add Medicine page
app.get('/add-medicine', requireAuth('admin'), async (req, res) => {
    try {
        const Hospital = require('./Hospital.js');
        const html = Hospital.getAddMedicineHTML();
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading add medicine:', err);
        res.status(500).send('Error loading page');
    }
});

// Add Lab page
app.get('/add-lab', requireAuth('admin'), async (req, res) => {
    try {
        const Hospital = require('./Hospital.js');
        const html = Hospital.getAddLabHTML();
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
```

```
        console.error('Error loading add lab:', err);
        res.status(500).send('Error loading page');
    }
});

// Login
app.post('/api/auth/login', async (req, res) => {
  try {
    const { username, email, password, role } = req.body;

    const result = await query(
      'SELECT * FROM users WHERE username = $1 OR email = $2',
      [username || '', email || '']
    );

    const user = result.rows[0];
    if (!user) {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    const isValid = await bcrypt.compare(password, user.password_hash);
    if (!isValid) {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    if (role && user.role !== role) {
      return res.status(403).json({ success: false, message: 'Wrong role' });
    }

    // Update last login
    await query('UPDATE users SET last_login = CURRENT_TIMESTAMP WHERE user_id =
$1', [user.user_id]);

    const token = generateToken({
      id: user.user_id,
      username: user.username,
      role: user.role
    });

    activeSessions.set(user.user_id, { token, loginTime: new Date().toISOString() });

    res.cookie('token', token, { httpOnly: true, maxAge: 7 * 24 * 60 * 60 * 1000 });

    res.json({
      success: true,
      token,
      user: { id: user.user_id, username: user.username, email: user.email, role: user.role }
    });
```

```
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server error' });
  }
});

// Logout
app.post('/api/auth/logout', (req, res) => {
    const token = req.cookies.token;
    if (token) {
        const decoded = verifyToken(token);
        if (decoded) activeSessions.delete(decoded.id);
    }
    res.clearCookie('token');
    res.json({ success: true, message: 'Logged out' });
});

// Get current user
app.get('/api/auth/me', authenticate, async (req, res) => {
  try {
    const result = await query(
      `SELECT u.user_id, u.username, u.email, u.role, u.created_at,
              CASE
                WHEN u.role = 'patient' THEN row_to_json(p.*)
                WHEN u.role = 'doctor' THEN row_to_json(d.*)
                WHEN u.role = 'lab' THEN row_to_json(l.*)
                WHEN u.role = 'admin' THEN row_to_json(a.*)
              END as profile
       FROM users u
       LEFT JOIN patients p ON u.user_id = p.user_id
       LEFT JOIN doctors d ON u.user_id = d.user_id
       LEFT JOIN lab_technicians l ON u.user_id = l.user_id
       LEFT JOIN hospital_admins a ON u.user_id = a.user_id
       WHERE u.user_id = $1`,
      [req.user.id]
    );

    if (!result.rows[0]) {
      return res.status(404).json({ success: false, message: 'User not found' });
    }

    res.json({ success: true, user: result.rows[0] });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server error' });
  }
});

// Patient API Routes
app.get('/api/patient', authenticate, authorize('patient'), async (req, res) => {
```

```
  try {
    const result = await query(
      `SELECT p.* FROM patients p
       JOIN users u ON p.user_id = u.user_id
       WHERE u.user_id = $1`,
      [req.user.id]
    );
    res.json(result.rows[0]);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Get patient history for lab technician
app.get('/api/patient-history', authenticate, authorize('lab'), async (req, res) => {
    try {
        // Get lab technician ID
        const labTechResult = await query(
            'SELECT lab_tech_id FROM lab_technicians WHERE user_id = $1',
            [req.user.id]
        );

        if (labTechResult.rows.length === 0) {
            return res.status(404).json({ success: false, message: 'Lab technician not found' });
        }

        const labTechId = labTechResult.rows[0].lab_tech_id;

        const result = await query(
            `SELECT
                lr.report_id,
                lr.report_uuid as id,
                p.full_name as name,
                p.patient_uuid as patient_id,
                lr.test_type,
                lr.test_date as date,
                lr.status,
                lr.priority,
                lr.created_at,
                d.doctor_uuid as doctor_id,
                d.full_name as doctor_name
             FROM lab_reports lr
             JOIN patients p ON lr.patient_id = p.patient_id
             LEFT JOIN doctors d ON lr.doctor_id = d.doctor_id
             WHERE lr.lab_tech_id = $1
             ORDER BY lr.created_at DESC
             LIMIT 50`,
            [labTechId]
```

```
      );

      res.json(result.rows);
    } catch (error) {
      console.error('Error fetching patient history:', error);
      res.status(500).json({ success: false, message: error.message });
    }
});

app.get('/api/appointments', authenticate, async (req, res) => {
  try {
    let result;
    if (req.user.role === 'patient') {
      result = await query(
        `SELECT a.*, d.full_name as doctor_name, d.specialization
         FROM appointments a
         JOIN doctors d ON a.doctor_id = d.doctor_id
         JOIN patients p ON a.patient_id = p.patient_id
         JOIN users u ON p.user_id = u.user_id
         WHERE u.user_id = $1
         ORDER BY a.appointment_date DESC`,
        [req.user.id]
      );
    } else if (req.user.role === 'doctor') {
      result = await query(
        `SELECT a.*, p.full_name as patient_name
         FROM appointments a
         JOIN patients p ON a.patient_id = p.patient_id
         JOIN doctors d ON a.doctor_id = d.doctor_id
         JOIN users u ON d.user_id = u.user_id
         WHERE u.user_id = $1 AND a.appointment_date = CURRENT_DATE
         ORDER BY a.appointment_time`,
        [req.user.id]
      );
    }
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.get('/api/doctors', authenticate, async (req, res) => {
  try {
    const result = await query(
      `SELECT d.*, h.name as hospital_name
       FROM doctors d
       JOIN hospitals h ON d.hospital_id = h.hospital_id
       WHERE d.status NOT IN ('On Leave', 'Inactive')
```

```
      ORDER BY d.full_name`
    );
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.get('/api/hospitals', authenticate, async (req, res) => {
  try {
    const result = await query('SELECT * FROM hospitals ORDER BY name');
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.get('/api/reports', authenticate, async (req, res) => {
  try {
    let result;
    if (req.user.role === 'patient') {
      result = await query(
        `SELECT r.* FROM lab_reports r
         JOIN patients p ON r.patient_id = p.patient_id
         JOIN users u ON p.user_id = u.user_id
         WHERE u.user_id = $1
         ORDER BY r.created_at DESC`,
        [req.user.id]
      );
    } else {
      result = await query(
        `SELECT r.*, p.full_name as patient_name
         FROM lab_reports r
         JOIN patients p ON r.patient_id = p.patient_id
         ORDER BY r.created_at DESC`
      );
    }
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.get('/api/prescriptions', authenticate, async (req, res) => {
  try {
    let result;
    if (req.user.role === 'patient') {
      result = await query(
```

```
        `SELECT p.* FROM prescriptions p
         JOIN patients pt ON p.patient_id = pt.patient_id
         JOIN users u ON pt.user_id = u.user_id
         WHERE u.user_id = $1 AND p.status = 'active'
         ORDER BY p.created_at DESC`,
        [req.user.id]
      );
    }
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// API endpoint for signin (used by signin.js)
app.post('/api/signin', async (req, res) => {
  try {
    const { username, password, role } = req.body;

    const result = await query(
      'SELECT * FROM users WHERE username = $1 OR email = $1',
      [username]
    );

    const user = result.rows[0];
    if (!user) {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    const isValid = await bcrypt.compare(password, user.password_hash);
    if (!isValid) {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    if (role && user.role !== role) {
      return res.status(403).json({ success: false, message: 'Wrong role' });
    }

    // Update last login
    await query('UPDATE users SET last_login = CURRENT_TIMESTAMP WHERE user_id =
$1', [user.user_id]);

    const token = generateToken({
      id: user.user_id,
      username: user.username,
      role: user.role
    });
```

```
    activeSessions.set(user.user_id, { token, loginTime: new Date().toISOString() });

    res.cookie('token', token, { httpOnly: true, maxAge: 7 * 24 * 60 * 60 * 1000 });

    // Determine redirect URL
    let redirectUrl = null;
    if (user.role === 'admin') {
        redirectUrl = '/admin-dashboard';
    } else if (user.role === 'doctor') {
        redirectUrl = '/doctor-dashboard';
    } else if (user.role === 'lab') {
        redirectUrl = '/lab-dashboard';
    } else if (user.role === 'patient') {
        redirectUrl = '/patient-dashboard';
    }

    res.json({
      success: true,
      message: 'Sign in successful!',
      user: {
        id: user.user_id,
        username: user.username,
        email: user.email,
        role: user.role
      },
      redirectTo: redirectUrl
    });
  } catch (error) {
    console.error('Sign in error:', error);
    res.status(500).json({ success: false, message: 'Server error' });
  }
});

app.post('/api/appointments', authenticate, authorize('patient'), async (req, res) => {
  try {
    const { doctor_id, appointment_date, appointment_time, reason, type, location } =
req.body;

    // Get patient_id from user
    const patientResult = await query(
      'SELECT patient_id FROM patients WHERE user_id = $1',
      [req.user.id]
    );

    const result = await query(
      `INSERT INTO appointments (
        patient_id, doctor_id, hospital_id, appointment_date,
        appointment_time, reason, type, location, status
```

```
      ) VALUES (
        $1, $2, (SELECT hospital_id FROM doctors WHERE doctor_id = $2),
        $3, $4, $5, $6, $7, 'pending'
      ) RETURNING *`,
      [patientResult.rows[0].patient_id, doctor_id, appointment_date,
       appointment_time, reason, type, location]
    );

    res.status(201).json(result.rows[0]);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// ==============================================
// APPOINTMENT RESCHEDULE & CANCEL ROUTES (FIXED with type casting)
// ==============================================

// Reschedule appointment
app.put('/api/appointments/:id/reschedule', authenticate, async (req, res) => {
  const client = await getClient();
  try {
    await client.query('BEGIN');

    const { id } = req.params;
    const { new_date, new_time, reason } = req.body;

    // Check if id is a UUID format or a string
    const isUUID = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i.test(id);

    let appointment;

    if (isUUID) {
      // It's a UUID, search by appointment_id
      appointment = await client.query(
        `SELECT a.*, p.user_id as patient_user_id, d.user_id as doctor_user_id
         FROM appointments a
         JOIN patients p ON a.patient_id = p.patient_id
         JOIN doctors d ON a.doctor_id = d.doctor_id
         WHERE a.appointment_id = $1::uuid`,
        [id]
      );
    } else {
      // It's a string like 'APT-001', search by appointment_uuid
      appointment = await client.query(
        `SELECT a.*, p.user_id as patient_user_id, d.user_id as doctor_user_id
         FROM appointments a
         JOIN patients p ON a.patient_id = p.patient_id
```

```
    JOIN doctors d ON a.doctor_id = d.doctor_id
    WHERE a.appointment_uuid = $1`,
   [id]
  );
}

if (!appointment.rows[0]) {
  await client.query('ROLLBACK');
  return res.status(404).json({ success: false, message: 'Appointment not found' });
}

const apt = appointment.rows[0];

// Check authorization (only patient or doctor can reschedule)
if (req.user.role === 'patient' && apt.patient_user_id !== req.user.id) {
  await client.query('ROLLBACK');
  return res.status(403).json({ success: false, message: 'Unauthorized' });
}

if (req.user.role === 'doctor' && apt.doctor_user_id !== req.user.id) {
  await client.query('ROLLBACK');
  return res.status(403).json({ success: false, message: 'Unauthorized' });
}

// Check if new slot is available - with explicit type casting
const slotCheck = await client.query(
  `SELECT * FROM appointments
   WHERE doctor_id = $1::uuid
   AND appointment_date = $2::date
   AND appointment_time = $3::varchar
   AND status != 'cancelled'
   AND appointment_id != $4::uuid`,
  [apt.doctor_id, new_date, new_time, apt.appointment_id]
);

if (slotCheck.rows.length > 0) {
  await client.query('ROLLBACK');
  return res.json({ success: false, message: 'Time slot not available' });
}

// Update appointment - with explicit type casting
await client.query(
  `UPDATE appointments
   SET appointment_date = $1::date,
       appointment_time = $2::varchar,
       notes = CASE WHEN $3::text IS NOT NULL AND $3::text != ''
               THEN COALESCE(notes, '') || ' | Rescheduled: ' || $3::text
               ELSE notes
```

```
                END,
        status = 'rescheduled'
      WHERE appointment_id = $4::uuid`,
     [new_date, new_time, reason, apt.appointment_id]
    );

    await client.query('COMMIT');

    res.json({
      success: true,
      message: 'Appointment rescheduled successfully'
    });

  } catch (error) {
    await client.query('ROLLBACK');
    console.error('Reschedule error:', error);
    res.status(500).json({ success: false, message: error.message });
  } finally {
    client.release();
  }
});

// Cancel appointment
app.put('/api/appointments/:id/cancel', authenticate, async (req, res) => {
  const client = await getClient();
  try {
    await client.query('BEGIN');

    const { id } = req.params;
    const { reason } = req.body;

    // Check if id is a UUID format or a string
    const isUUID = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i.test(id);

    let appointment;

    if (isUUID) {
      // It's a UUID, search by appointment_id
      appointment = await client.query(
        `SELECT a.*, p.user_id as patient_user_id, d.user_id as doctor_user_id
         FROM appointments a
         JOIN patients p ON a.patient_id = p.patient_id
         JOIN doctors d ON a.doctor_id = d.doctor_id
         WHERE a.appointment_id = $1::uuid`,
        [id]
      );
    } else {
      // It's a string like 'APT-001', search by appointment_uuid
```

```
    appointment = await client.query(
      `SELECT a.*, p.user_id as patient_user_id, d.user_id as doctor_user_id
       FROM appointments a
       JOIN patients p ON a.patient_id = p.patient_id
       JOIN doctors d ON a.doctor_id = d.doctor_id
       WHERE a.appointment_uuid = $1`,
      [id]
    );
  }

  if (!appointment.rows[0]) {
    await client.query('ROLLBACK');
    return res.status(404).json({ success: false, message: 'Appointment not found' });
  }

  const apt = appointment.rows[0];

  // Check authorization
  if (req.user.role === 'patient' && apt.patient_user_id !== req.user.id) {
    await client.query('ROLLBACK');
    return res.status(403).json({ success: false, message: 'Unauthorized' });
  }

  if (req.user.role === 'doctor' && apt.doctor_user_id !== req.user.id) {
    await client.query('ROLLBACK');
    return res.status(403).json({ success: false, message: 'Unauthorized' });
  }

  // Update appointment status to cancelled - with explicit type casting
  await client.query(
    `UPDATE appointments
     SET status = 'cancelled',
         notes = CASE WHEN $1::text IS NOT NULL AND $1::text != ''
                 THEN COALESCE(notes, '') || ' | Cancelled: ' || $1::text
                 ELSE COALESCE(notes, '') || ' | Cancelled by patient'
                 END
     WHERE appointment_id = $2::uuid`,
    [reason, apt.appointment_id]
  );

  await client.query('COMMIT');

  res.json({
    success: true,
    message: 'Appointment cancelled successfully'
  });

} catch (error) {
```

```
    await client.query('ROLLBACK');
    console.error('Cancel error:', error);
    res.status(500).json({ success: false, message: error.message });
  } finally {
    client.release();
  }
});

app.put('/api/patient', authenticate, authorize('patient'), async (req, res) => {
  try {
    const result = await query(
      `UPDATE patients
       SET full_name = COALESCE($1, full_name),
           phone = COALESCE($2, phone),
           address = COALESCE($3, address),
           emergency_contact_name = COALESCE($4, emergency_contact_name),
           emergency_contact_phone = COALESCE($5, emergency_contact_phone)
       WHERE user_id = $6
       RETURNING *`,
      [
        req.body.full_name,
        req.body.phone,
        req.body.address,
        req.body.emergency_contact_name,
        req.body.emergency_contact_phone,
        req.user.id
      ]
    );
    res.json(result.rows[0]);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// SERVE PATIENT SIGNUP
// SERVE PATIENT SIGNUP - Add this near your other route definitions
// SERVE PATIENT SIGNUP - EXACT copy from signup_patient.js
// SERVE PATIENT SIGNUP - Using external file
app.get('/patient-signup', (req, res) => {
    try {
        const renderPatientSignup = require('./signup_patient.js');
        const html = renderPatientSignup();
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading signup_patient.js:', err);
        res.status(500).send(`
            <h1>500 - Patient Signup not found</h1>
```

```
            <p>Make sure signup_patient.js is in the same directory and exports the render
function.</p>
            <a href="/">← Back to Home</a>
        `);
    }
});

// Serve the main HTML page
app.get('/', (req, res) => {
    const html = generateHTML();
    res.send(html);
});

// Serve signin page
// Add this route to your main Express app
// Serve signin page - Using external file
app.get('/signin', (req, res) => {
    try {
        const renderSignInPage = require('./signin.js');
        const html = renderSignInPage();
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading signin.js:', err);
        res.status(500).send(`
            <h1>500 - Sign In page not found</h1>
            <p>Make sure signin.js is in the same directory and exports the render function.</p>
            <a href="/">← Back to Home</a>
        `);
    }
});
// ==============================================
// DOCTOR API ROUTES - Copied from Doctor.js
// ==============================================

// Doctor API Routes
app.get('/api/doctor', authenticate, authorize('doctor'), async (req, res) => {
  try {
    const result = await query(
      `SELECT d.*, h.name as hospital_name
       FROM doctors d
       JOIN hospitals h ON d.hospital_id = h.hospital_id
       WHERE d.user_id = $1`,
      [req.user.id]
    );
    res.json(result.rows[0]);
  } catch (error) {
    res.status(500).json({ error: error.message });
```

```
  }
});

app.get('/api/appointments/today', authenticate, authorize('doctor'), async (req, res) => {
  try {
    const result = await query(
      `SELECT a.*, p.full_name as patient_name
       FROM appointments a
       JOIN patients p ON a.patient_id = p.patient_id
       JOIN doctors d ON a.doctor_id = d.doctor_id
       WHERE d.user_id = $1 AND a.appointment_date = CURRENT_DATE
       ORDER BY a.appointment_time`,
      [req.user.id]
    );
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});



app.get('/api/lab-reports', authenticate, authorize('doctor'), async (req, res) => {
  try {
    const result = await query(
      `SELECT r.*, p.full_name as patient_name
       FROM lab_reports r
       JOIN patients p ON r.patient_id = p.patient_id
       JOIN doctors d ON r.doctor_id = d.doctor_id
       WHERE d.user_id = $1 AND r.status = 'pending'
       ORDER BY r.created_at DESC`,
      [req.user.id]
    );
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.get('/api/patients', authenticate, authorize('doctor'), async (req, res) => {
  try {
    const result = await query(
      `SELECT DISTINCT p.*
       FROM patients p
       JOIN appointments a ON p.patient_id = a.patient_id
       JOIN doctors d ON a.doctor_id = d.doctor_id
       WHERE d.user_id = $1
       ORDER BY p.full_name`,
```

```
      [req.user.id]
    );
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// SERVE HOSPITAL DASHBOARD - Using Hospital.js
app.get('/hospital-dashboard', requireAuth('admin'), async (req, res) => {
    try {
        // Get hospital data for the logged-in admin
        const hospitalResult = await query(
            `SELECT h.* FROM hospitals h
             JOIN hospital_admins ha ON h.hospital_id = ha.hospital_id
             WHERE ha.user_id = $1`,
            [req.user.id]
        );

        // Get dashboard statistics
        const statsResult = await query(
            `SELECT
                (SELECT COUNT(*) FROM doctors WHERE hospital_id = $1) as total_doctors,
                (SELECT COUNT(*) FROM appointments WHERE hospital_id = $1 AND
appointment_date >= CURRENT_DATE) as upcoming_appointments,
                (SELECT COUNT(*) FROM medicines WHERE hospital_id = $1) as
total_medicines,
                (SELECT COUNT(*) FROM lab_technicians WHERE hospital_id = $1) as
total_labs`,
            [hospitalResult.rows[0]?.hospital_id || null]
        );

        // Get recent activity
        const recentResult = await query(
            `SELECT 'appointment' as type, a.appointment_date as date,
                    p.full_name as patient_name, d.full_name as doctor_name
             FROM appointments a
             JOIN patients p ON a.patient_id = p.patient_id
             JOIN doctors d ON a.doctor_id = d.doctor_id
             WHERE a.hospital_id = $1
             ORDER BY a.created_at DESC
             LIMIT 10`,
            [hospitalResult.rows[0]?.hospital_id || null]
        );

        // Require and render the dashboard
        const Hospital = require('./Hospital.js');
```

```
        // Pass data to the render function
        const html = await Hospital.renderHospitalDashboard({
            hospital: hospitalResult.rows[0] || null,
            stats: statsResult.rows[0] || {},
            recentActivity: recentResult.rows || []
        });

        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading Hospital.js:', err);
        res.status(500).send(`
            <!DOCTYPE html>
            <html>
            <head>
                <title>Error - Hospital Dashboard</title>
                <style>
                    body { font-family: Arial, sans-serif; padding: 40px; background: #f5f5f5;
text-align: center; }
                    .error-container { max-width: 600px; margin: 0 auto; background: white;
padding: 40px; border-radius: 10px; box-shadow: 0 2px 10px rgba(0,0,0,0.1); }
                    h1 { color: #e53e3e; }
                    .back-btn { display: inline-block; margin-top: 20px; padding: 10px 20px;
background: #4299e1; color: white; text-decoration: none; border-radius: 5px; }
                </style>
            </head>
            <body>
                <div class="error-container">
                    <h1>500 - Hospital Dashboard Error</h1>
                    <p>${err.message}</p>
                    <p>Make sure Hospital.js is in the same directory.</p>
                    <a href="/signin" class="back-btn">← Back to Sign In</a>
                </div>
            </body>
            </html>
        `);
    }
});

app.put('/api/doctor', authenticate, authorize('doctor'), async (req, res) => {
  try {
    const result = await query(
      `UPDATE doctors
       SET full_name = COALESCE($1, full_name),
           designation = COALESCE($2, designation),
           consultation_fee = COALESCE($3, consultation_fee),
           available_days = COALESCE($4, available_days),
           available_time = COALESCE($5, available_time),
```

```
        phone = COALESCE($6, phone),
        address = COALESCE($7, address)
      WHERE user_id = $8
      RETURNING *`,
     [
       req.body.full_name,
       req.body.designation,
       req.body.consultation_fee,
       req.body.available_days,
       req.body.available_time,
       req.body.phone,
       req.body.address,
       req.user.id
     ]
   );
   res.json(result.rows[0]);
 } catch (error) {
   res.status(500).json({ error: error.message });
 }
});

app.put('/api/appointments/:id', authenticate, authorize('doctor', 'admin'), async (req, res) => {
 try {
   const result = await query(
     `UPDATE appointments
      SET status = COALESCE($1, status),
        appointment_time = COALESCE($2, appointment_time),
        reason = COALESCE($3, reason),
        notes = COALESCE($4, notes)
      WHERE appointment_id = $5
      RETURNING *`,
     [
       req.body.status,
       req.body.appointment_time,
       req.body.reason,
       req.body.notes,
       req.params.id
     ]
   );

   if (result.rows.length === 0) {
     return res.status(404).json({ error: 'Appointment not found' });
   }

   res.json(result.rows[0]);
 } catch (error) {
   res.status(500).json({ error: error.message });
 }
```

```
});

// API endpoint for feedback submission
app.post('/api/feedback', async (req, res) => {
  try {
    const { name, email, message } = req.body;
    await query(
      'INSERT INTO feedback (name, email, message) VALUES ($1, $2, $3)',
      [name, email, message]
    );
    res.json({ success: true, message: 'Thank you for your feedback!' });
  } catch (error) {
    res.status(500).json({ success: false, error: error.message });
  }
});

// API endpoint for signin
//app.post('/api/signin', (req, res) => {
  //  console.log('Sign in attempt:', req.body);
    // In real app: validate credentials against database
    //res.json({
      //  success: true,
        //message: 'Sign in successful!',
        //user: { name: req.body.email.split('@')[0] }
    //});
//});

// Serve static SDK files (simulated)
app.get('/_sdk/element_sdk.js', (req, res) => {
    res.send(`
        // Mock SDK
        window.elementSdk = {
            init: function(config) {
                console.log('SDK initialized', config);
                return this;
            },
            setConfig: function(config) {
                console.log('Config updated', config);
            }
        };
    `);
});

app.get('/_sdk/data_sdk.js', (req, res) => {
    res.send(`
        // Mock Data SDK
        console.log('Data SDK loaded');
    `);
```

```
});

function generateHTML() {
    return `
<!doctype html>
<html lang="en" class="h-full">
 <head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>BondHealth - Unifying Care, Strengthening Lives</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="/_sdk/element_sdk.js"></script>
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700;800&a
mp;display=swap" rel="stylesheet">
  <style>
    body {
      box-sizing: border-box;
    }
    * {
      font-family: 'Poppins', sans-serif;
    }

    @keyframes slideUp {
      from { opacity: 0; transform: translateY(30px); }
      to { opacity: 1; transform: translateY(0); }
    }

    @keyframes countUp {
      from { opacity: 0; transform: scale(0.8); }
      to { opacity: 1; transform: scale(1); }
    }

    @keyframes float {
      0%, 100% { transform: translateY(0) rotate(0deg); }
      50% { transform: translateY(-20px) rotate(5deg); }
    }

    @keyframes pulse-glow {
      0%, 100% { box-shadow: 0 0 30px rgba(0, 200, 255, 0.5); }
      50% { box-shadow: 0 0 60px rgba(0, 200, 255, 0.8); }
    }

    @keyframes drift {
      0%, 100% { transform: translate(0, 0); }
      25% { transform: translate(20px, -20px); }
      50% { transform: translate(-20px, 20px); }
      75% { transform: translate(20px, 10px); }
```

```
}

@keyframes spin-slow {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}

@keyframes bounce-subtle {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-5px); }
}

.animate-slide-up {
  animation: slideUp 0.8s ease-out forwards;
}

.animate-float {
  animation: float 3s ease-in-out infinite;
}

.animate-pulse-glow {
  animation: pulse-glow 2s ease-in-out infinite;
}

.stat-card {
  animation: countUp 0.6s ease-out forwards;
}

.stat-card:nth-child(1) { animation-delay: 0.1s; }
.stat-card:nth-child(2) { animation-delay: 0.2s; }
.stat-card:nth-child(3) { animation-delay: 0.3s; }
.stat-card:nth-child(4) { animation-delay: 0.4s; }

.review-bubble {
  position: relative;
  transition: all 0.3s ease;
}

.review-bubble:hover {
  transform: translateY(-4px);
  box-shadow: 0 12px 30px rgba(0, 86, 179, 0.15);
}

.gradient-text {
  background: linear-gradient(135deg, #00c8ff, #10ebebdc);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
```

```
}

.hero-gradient {
  background: linear-gradient(135deg, #53f8f8 0%, #ffffff 50%, #e0ffff 100%);
}

.floating-shape {
  position: absolute;
  border-radius: 50%;
  opacity: 0.15;
  animation: drift 15s infinite ease-in-out;
}

.rotating-ring {
  animation: spin-slow 20s linear infinite;
}

.card-hover {
  transition: all 0.3s ease;
}

.card-hover:hover {
  transform: translateY(-8px);
  box-shadow: 0 20px 40px rgba(0, 86, 179, 0.15);
}

.slider-track {
  display: flex;
  animation: slide 20s linear infinite;
}

@keyframes slide {
  0% { transform: translateX(0); }
  100% { transform: translateX(-50%); }
}

.reviews-container {
  max-height: 600px;
  overflow-y: auto;
  scrollbar-width: thin;
  scrollbar-color: #0056b3 #e6f2ff;
  padding-right: 8px;
}

.reviews-container::-webkit-scrollbar {
  width: 6px;
}
```

```
    .reviews-container::-webkit-scrollbar-track {
      background: #e6f2ff;
      border-radius: 3px;
    }

    .reviews-container::-webkit-scrollbar-thumb {
      background: #0056b3;
      border-radius: 3px;
    }

    .rating-stars {
      display: inline-flex;
      gap: 2px;
    }

    .rating-stars .star {
      color: #d1d5db;
      transition: color 0.2s;
    }

    .rating-stars .star.filled {
      color: #fbbf24;
    }

    .filter-btn.active {
      background: linear-gradient(135deg, #0088cc, #00aadd);
      color: white;
      border-color: transparent;
    }

    .loading-spinner {
      border: 3px solid #f3f3f3;
      border-top: 3px solid #0088cc;
      border-radius: 50%;
      width: 40px;
      height: 40px;
      animation: spin 1s linear infinite;
    }

    @keyframes spin {
      0% { transform: rotate(0deg); }
      100% { transform: rotate(360deg); }
    }
  </style>
  <style>@view-transition { navigation: auto; }</style>
  <script src="/_sdk/data_sdk.js" type="text/javascript"></script>
 </head>
 <body class="h-full bg-white">
```

```
  <div id="app-wrapper" class="w-full h-full overflow-auto">
   <!-- Header -->
   <header class="sticky top-0 z-50 bg-white/95 backdrop-blur-sm shadow-sm">
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
     <div class="flex justify-between items-center py-4">
      <div class="flex items-center gap-2">
       <svg class="w-10 h-10" viewbox="0 0 50 50" fill="none">
        <circle cx="25" cy="25" r="23" fill="#0088cc" opacity="0.2" />
        <path d="M25 8C15.6 8 8 15.6 8 25s7.6 17 17 17 17-7.6 17-17S34.4 8 25 8z"
stroke="#0088cc" stroke-width="2" fill="none" />
        <path d="M25 15v20M15 25h20" stroke="#0088cc" stroke-width="3"
stroke-linecap="round" />
        <circle cx="25" cy="25" r="6" fill="#00aadd" opacity="0.4" />
       </svg>
       <span class="text-2xl font-bold text-[#006d77]">Bond<span
class="text-[#00c8ff]">Health</span></span>
      </div>
      <nav class="hidden md:flex items-center gap-8">
        <a href="#about" class="text-gray-600 hover:text-[#00c8ff] transition-all font-medium
hover:scale-110">About</a>
        <a href="#network" class="text-gray-600 hover:text-[#00c8ff] transition-all font-medium
hover:scale-110">Network</a>
        <a href="#hospitals" class="text-gray-600 hover:text-[#00c8ff] transition-all font-medium
hover:scale-110">Hospitals</a>
        <a href="#reviews" class="text-gray-600 hover:text-[#00c8ff] transition-all font-medium
hover:scale-110">Reviews</a>
      </nav>
      <button onclick="window.location.href='/signin'" class="bg-gradient-to-r from-[#00c8ff]
to-[#00ffff] hover:from-[#00b0e0] hover:to-[#00e0e0] text-white px-6 py-2.5 rounded-full
font-medium transition-all hover:shadow-lg hover:shadow-cyan-300 hover:scale-105"> Sign
In </button>
     </div>
    </div>
   </header>

   <!-- Hero Section -->
   <section class="hero-gradient py-20 lg:py-28 relative overflow-hidden">
    <div class="floating-shape top-20 right-10 w-64 h-64 bg-[#0088cc]"
style="animation-delay: 0s;"></div>
    <div class="floating-shape bottom-10 left-10 w-96 h-96 bg-[#00aadd]"
style="animation-delay: 2s;"></div>
    <div class="floating-shape top-40 left-1/4 w-48 h-48 bg-[#0099cc]" style="animation-delay:
4s;"></div>
    <div class="floating-shape bottom-40 right-1/3 w-72 h-72 bg-[#66ccff]"
style="animation-delay: 6s;"></div>
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 relative">
     <div class="grid lg:grid-cols-2 gap-12 items-center">
      <div class="animate-slide-up">
```

<h1 id="tagline" class="text-4xl md:text-5xl lg:text-6xl font-bold text-[#006d77] leading-tight mb-6 animate-slide-up">Unifying Care,
Strengthening Lives.</h1>

BondHealth is a state-wide digital ecosystem that bridges the gap between you and your medical providers. We securely centralize your health records, ensuring that your medical history, appointments, and lab results are accessible at every hospital you visit.

<button onclick="window.location.href='/signin'" class="bg-gradient-to-r from-[#00c8ff] to-[#00ffff] hover:from-[#00b0e0] hover:to-[#00e0e0] text-white px-8 py-3.5 rounded-full font-semibold transition-all hover:shadow-xl hover:shadow-cyan-300 animate-pulse-glow hover:scale-105"> Get Started Free </button>

<button onclick="window.location.href='/signin'" class="border-2 border-[#00c8ff] text-[#00c8ff] hover:bg-[#00c8ff] hover:text-white px-8 py-3.5 rounded-full font-semibold transition-all hover:scale-105 hover:shadow-lg"> Learn More </button>

<svg class="w-full max-w-lg mx-auto" viewbox="0 0 400 350" fill="none">

<circle class="rotating-ring" cx="200" cy="175" r="60" fill="#0088cc" opacity="0.1" />

<circle cx="200" cy="175" r="45" fill="#0088cc" opacity="0.2" style="animation: pulse-glow 3s ease-in-out infinite;" />

<circle cx="200" cy="175" r="30" fill="url(#cyan-gradient)" />

<path d="M200 155v40M180 175h40" stroke="white" stroke-width="4" stroke-linecap="round" />

<defs>

<lineargradient id="cyan-gradient" x1="0%" y1="0%" x2="100%" y2="100%">

<stop offset="0%" style="stop-color:#0088cc;stop-opacity:1" />

<stop offset="100%" style="stop-color:#00aadd;stop-opacity:1" />

</lineargradient>

</defs>

<circle cx="80" cy="100" r="25" fill="#0088cc" opacity="0.9" style="animation: bounce-subtle 2s ease-in-out infinite;">

<animate attributename="r" values="25;28;25" dur="2s" repeatcount="indefinite" />

</circle>

<text x="80" y="105" text-anchor="middle" fill="white" font-size="20">🏥</text>

<line x1="105" y1="115" x2="160" y2="155" stroke="#0088cc" stroke-width="2" stroke-dasharray="5,5" opacity="0.6">

<animate attributename="stroke-dashoffset" values="0;-10" dur="1s" repeatcount="indefinite" />

</line>

<circle cx="320" cy="100" r="25" fill="#0099cc" opacity="0.9" style="animation: bounce-subtle 2s ease-in-out infinite; animation-delay: 0.5s;">

<animate attributename="r" values="25;28;25" dur="2s" repeatcount="indefinite" begin="0.5s" />

</circle>

<text x="320" y="105" text-anchor="middle" fill="white" font-size="20">👨‍⚕️ </text>

```
        <line x1="295" y1="115" x2="240" y2="155" stroke="#0099cc" stroke-width="2"
stroke-dasharray="5,5" opacity="0.6">
          <animate attributename="stroke-dashoffset" values="0;-10" dur="1s"
repeatcount="indefinite" />
        </line>
        <circle cx="80" cy="250" r="25" fill="#00aadd" opacity="0.9" style="animation:
bounce-subtle 2s ease-in-out infinite; animation-delay: 1s;">
          <animate attributename="r" values="25;28;25" dur="2s" repeatcount="indefinite"
begin="1s" />
        </circle>
        <text x="80" y="255" text-anchor="middle" fill="white" font-size="20">🔬</text>
        <line x1="105" y1="235" x2="160" y2="195" stroke="#00aadd" stroke-width="2"
stroke-dasharray="5,5" opacity="0.6">
          <animate attributename="stroke-dashoffset" values="0;-10" dur="1s"
repeatcount="indefinite" />
        </line>
        <circle cx="320" cy="250" r="25" fill="#66ccff" opacity="0.9" style="animation:
bounce-subtle 2s ease-in-out infinite; animation-delay: 1.5s;">
          <animate attributename="r" values="25;28;25" dur="2s" repeatcount="indefinite"
begin="1.5s" />
        </circle>
        <text x="320" y="255" text-anchor="middle" fill="white" font-size="20">👥</text>
        <line x1="295" y1="235" x2="240" y2="195" stroke="#66ccff" stroke-width="2"
stroke-dasharray="5,5" opacity="0.6">
          <animate attributename="stroke-dashoffset" values="0;-10" dur="1s"
repeatcount="indefinite" />
        </line>
       </svg>
      </div>
     </div>
    </div>
   </section>

   <!-- Impact Slider Section -->
   <section id="network" class="py-20 bg-white">
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
     <div class="text-center mb-12">
      <h2 class="text-3xl md:text-4xl font-bold text-[#1a365d] mb-4">The Bond Network</h2>
      <p class="text-gray-600 max-w-2xl mx-auto">Our growing ecosystem connects
healthcare providers across the state</p>
     </div>
     <div class="grid grid-cols-2 lg:grid-cols-4 gap-6">
      <div class="stat-card bg-gradient-to-br from-blue-50 to-white p-6 rounded-2xl shadow-lg
border border-blue-100 card-hover text-center hover:shadow-blue-200">
       <div class="text-4xl mb-3 animate-bounce-subtle" style="animation-duration:
3s;">🏥</div>
       <div class="text-3xl md:text-4xl font-bold text-[#0088cc] mb-1"
data-count="120">120+</div>
```

```
      <div class="text-sm text-gray-600 font-medium">Partner Hospitals</div>
      <div class="text-xs text-gray-400 mt-1">Across the State</div>
     </div>
     <div class="stat-card bg-gradient-to-br from-cyan-50 to-white p-6 rounded-2xl shadow-lg
border border-cyan-100 card-hover text-center hover:shadow-cyan-200">
      <div class="text-4xl mb-3 animate-bounce-subtle" style="animation-duration: 3.2s;
animation-delay: 0.2s;">👥</div>
      <div class="text-3xl md:text-4xl font-bold text-[#0099cc] mb-1"
data-count="50000">50,000+</div>
      <div class="text-sm text-gray-600 font-medium">Active Users</div>
      <div class="text-xs text-gray-400 mt-1">Lives Connected</div>
     </div>
     <div class="stat-card bg-gradient-to-br from-teal-50 to-white p-6 rounded-2xl shadow-lg
border border-teal-100 card-hover text-center hover:shadow-teal-200">
      <div class="text-4xl mb-3 animate-bounce-subtle" style="animation-duration: 3.4s;
animation-delay: 0.4s;">👨‍⚕️</div>
      <div class="text-3xl md:text-4xl font-bold text-[#00aadd] mb-1"
data-count="1500">1,500+</div>
      <div class="text-sm text-gray-600 font-medium">Verified Doctors</div>
      <div class="text-xs text-gray-400 mt-1">Specialists On-call</div>
     </div>
     <div class="stat-card bg-gradient-to-br from-sky-50 to-white p-6 rounded-2xl shadow-lg
border border-sky-100 card-hover text-center hover:shadow-sky-200">
      <div class="text-4xl mb-3 animate-bounce-subtle" style="animation-duration: 3.6s;
animation-delay: 0.6s;">🔬</div>
      <div class="text-3xl md:text-4xl font-bold text-[#66ccff] mb-1"
data-count="300">300+</div>
      <div class="text-sm text-gray-600 font-medium">Diagnostic Labs</div>
      <div class="text-xs text-gray-400 mt-1">Instant Results</div>
     </div>
    </div>
   </div>
  </section>

  <!-- Hospitals Gallery -->
  <section id="hospitals" class="py-20 bg-gradient-to-b from-gray-50 to-white">
   <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
    <div class="text-center mb-12">
     <h2 class="text-3xl md:text-4xl font-bold text-[#1a365d] mb-4">Frequently Visited
Hospitals</h2>
     <p class="text-gray-600 max-w-2xl mx-auto">Find and book appointments at our partner
facilities</p>
    </div>
    <div class="grid sm:grid-cols-2 lg:grid-cols-4 gap-6">
     <div class="bg-white rounded-2xl shadow-lg overflow-hidden card-hover
hover:shadow-blue-200">
      <div class="aspect-video bg-gradient-to-br from-[#0088cc] to-[#00aadd] flex
items-center justify-center relative overflow-hidden">
```

```
        <div class="absolute inset-0 bg-white opacity-20" style="animation: drift 10s infinite
ease-in-out;"></div>
        <svg class="w-20 h-20 text-white opacity-90 relative z-10" viewbox="0 0 24 24"
fill="currentColor" style="animation: bounce-subtle 3s infinite;"><path d="M19 3H5c-1.1 0-2
.9-2 2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9 2-2V5c0-1.1-.9-2-2-2zm-1
11h-4v4h-4v-4H6v-4h4V6h4v4h4v4z" />></svg>
       </div>
       <div class="p-5">
        <h3 class="font-bold text-[#006d77] text-lg">City General Hospital</h3>
        <p class="text-gray-500 text-sm mb-4">Downtown Branch</p>
        <button class="w-full bg-gradient-to-r from-[#0088cc] to-[#00aadd]
hover:from-[#0077bb] hover:to-[#0099cc] text-white py-2.5 rounded-lg font-medium
transition-all text-sm hover:shadow-lg hover:scale-105"> Select Facility </button>
       </div>
      </div>
      <div class="bg-white rounded-2xl shadow-lg overflow-hidden card-hover
hover:shadow-cyan-200">
       <div class="aspect-video bg-gradient-to-br from-[#0099cc] to-[#66ccff] flex items-center
justify-center relative overflow-hidden">
        <div class="absolute inset-0 bg-white opacity-20" style="animation: drift 12s infinite
ease-in-out;"></div>
        <svg class="w-20 h-20 text-white opacity-90 relative z-10" viewbox="0 0 24 24"
fill="currentColor" style="animation: bounce-subtle 3.2s infinite;"><path d="M19 3H5c-1.1 0-2
.9-2 2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9 2-2V5c0-1.1-.9-2-2-2zm-1
11h-4v4h-4v-4H6v-4h4V6h4v4h4v4z" />></svg>
       </div>
       <div class="p-5">
        <h3 class="font-bold text-[#006d77] text-lg">St. Mary's Medical</h3>
        <p class="text-gray-500 text-sm mb-4">Westside Campus</p>
        <button class="w-full bg-gradient-to-r from-[#0099cc] to-[#66ccff] hover:from-[#0088bb]
hover:to-[#55bbee] text-white py-2.5 rounded-lg font-medium transition-all text-sm
hover:shadow-lg hover:scale-105"> Select Facility </button>
       </div>
      </div>
      <div class="bg-white rounded-2xl shadow-lg overflow-hidden card-hover
hover:shadow-teal-200">
       <div class="aspect-video bg-gradient-to-br from-[#00aadd] to-[#00aadd] flex
items-center justify-center relative overflow-hidden">
        <div class="absolute inset-0 bg-white opacity-20" style="animation: drift 14s infinite
ease-in-out;"></div>
        <svg class="w-20 h-20 text-white opacity-90 relative z-10" viewbox="0 0 24 24"
fill="currentColor" style="animation: bounce-subtle 3.4s infinite;"><path d="M19 3H5c-1.1 0-2
.9-2 2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9 2-2V5c0-1.1-.9-2-2-2zm-1
11h-4v4h-4v-4H6v-4h4V6h4v4h4v4z" />></svg>
       </div>
       <div class="p-5">
        <h3 class="font-bold text-[#006d77] text-lg">Regional Health Center</h3>
        <p class="text-gray-500 text-sm mb-4">North District</p>
```

```
        <button class="w-full bg-gradient-to-r from-[#00aadd] to-[#00aadd]
hover:from-[#0099cc] hover:to-[#0099cc] text-white py-2.5 rounded-lg font-medium
transition-all text-sm hover:shadow-lg hover:scale-105"> Select Facility </button>
       </div>
      </div>
      <div class="bg-white rounded-2xl shadow-lg overflow-hidden card-hover
hover:shadow-sky-200">
       <div class="aspect-video bg-gradient-to-br from-[#66ccff] to-[#99ddff] flex items-center
justify-center relative overflow-hidden">
        <div class="absolute inset-0 bg-white opacity-20" style="animation: drift 16s infinite
ease-in-out;"></div>
        <svg class="w-20 h-20 text-white opacity-90 relative z-10" viewbox="0 0 24 24"
fill="currentColor" style="animation: bounce-subtle 3.6s infinite;"><path d="M19 3H5c-1.1 0-2
.9-2 2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9 2-2V5c0-1.1-.9-2-2-2zm-1
11h-4v4h-4v-4H6v-4h4V6h4v4h4v4z" />></svg>
       </div>
       <div class="p-5">
        <h3 class="font-bold text-[#006d77] text-lg">Unity Care Hospital</h3>
        <p class="text-gray-500 text-sm mb-4">East End Location</p>
        <button class="w-full bg-gradient-to-r from-[#66ccff] to-[#99ddff] hover:from-[#55bbee]
hover:to-[#88ccee] text-white py-2.5 rounded-lg font-medium transition-all text-sm
hover:shadow-lg hover:scale-105"> Book Now </button>
       </div>
      </div>
     </div>
    </div>
   </section>

   <!-- Reviews Section - Dynamic -->
   <section id="reviews" class="py-20 bg-white">
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
     <div class="text-center mb-8">
      <h2 class="text-3xl md:text-4xl font-bold text-[#1a365d] mb-4">What Our Users
Say</h2>
      <p class="text-gray-600 max-w-2xl mx-auto">Real experiences from patients across the
state</p>
     </div>

     <!-- Rating Summary -->
     <div id="rating-summary" class="bg-gradient-to-br from-blue-50 to-white p-6 rounded-2xl
shadow-lg border border-blue-100 mb-8">
       <div class="flex flex-col md:flex-row items-center gap-8">
         <div class="text-center">
           <div id="average-rating" class="text-5xl font-bold text-[#0088cc]">4.8</div>
           <div class="rating-stars text-2xl mt-2" id="average-stars">★★★★★</div>
           <div id="total-reviews" class="text-sm text-gray-600 mt-1">Based on 0 reviews</div>
         </div>
         <div class="flex-1 grid grid-cols-1 sm:grid-cols-5 gap-2 w-full">
```

```
          <div class="col-span-1 flex items-center gap-2">
            <span class="text-sm font-medium w-12">5 ★</span>
            <div class="flex-1 h-2 bg-gray-200 rounded-full overflow-hidden">
              <div id="five-star-bar" class="h-full bg-green-500" style="width: 0%"></div>
            </div>
            <span id="five-star-count" class="text-sm text-gray-600 w-10">0</span>
          </div>
          <div class="col-span-1 flex items-center gap-2">
            <span class="text-sm font-medium w-12">4 ★</span>
            <div class="flex-1 h-2 bg-gray-200 rounded-full overflow-hidden">
              <div id="four-star-bar" class="h-full bg-green-400" style="width: 0%"></div>
            </div>
            <span id="four-star-count" class="text-sm text-gray-600 w-10">0</span>
          </div>
          <div class="col-span-1 flex items-center gap-2">
            <span class="text-sm font-medium w-12">3 ★</span>
            <div class="flex-1 h-2 bg-gray-200 rounded-full overflow-hidden">
              <div id="three-star-bar" class="h-full bg-yellow-400" style="width: 0%"></div>
            </div>
            <span id="three-star-count" class="text-sm text-gray-600 w-10">0</span>
          </div>
          <div class="col-span-1 flex items-center gap-2">
            <span class="text-sm font-medium w-12">2 ★</span>
            <div class="flex-1 h-2 bg-gray-200 rounded-full overflow-hidden">
              <div id="two-star-bar" class="h-full bg-orange-400" style="width: 0%"></div>
            </div>
            <span id="two-star-count" class="text-sm text-gray-600 w-10">0</span>
          </div>
          <div class="col-span-1 flex items-center gap-2">
            <span class="text-sm font-medium w-12">1 ★</span>
            <div class="flex-1 h-2 bg-gray-200 rounded-full overflow-hidden">
              <div id="one-star-bar" class="h-full bg-red-500" style="width: 0%"></div>
            </div>
            <span id="one-star-count" class="text-sm text-gray-600 w-10">0</span>
          </div>
        </div>
      </div>
    </div>

    <!-- Filters and Sorting -->
    <div class="flex flex-col md:flex-row justify-between items-start md:items-center gap-4
mb-6">
      <div class="flex flex-wrap gap-2">
        <button class="filter-btn active px-4 py-2 rounded-full border border-gray-300 text-sm
font-medium transition-all" data-filter="all">All Reviews</button>
        <button class="filter-btn px-4 py-2 rounded-full border border-gray-300 text-sm
font-medium transition-all" data-filter="positive">Positive (4-5★)</button>
```

```
        <button class="filter-btn px-4 py-2 rounded-full border border-gray-300 text-sm
font-medium transition-all" data-filter="neutral">Neutral (3★)</button>
        <button class="filter-btn px-4 py-2 rounded-full border border-gray-300 text-sm
font-medium transition-all" data-filter="negative">Negative (1-2★)</button>
      </div>

      <div class="flex items-center gap-2 w-full md:w-auto">
        <select id="sort-select" class="px-4 py-2 rounded-lg border border-gray-300 text-sm
focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none">
          <option value="recent">Most Recent</option>
          <option value="highest">Highest Rated</option>
          <option value="lowest">Lowest Rated</option>
          <option value="helpful">Most Helpful</option>
        </select>

        <button id="write-review-btn" class="bg-gradient-to-r from-[#0088cc] to-[#00aadd]
text-white px-4 py-2 rounded-lg text-sm font-medium hover:shadow-lg transition-all
whitespace-nowrap">
          Write a Review
        </button>
      </div>
    </div>

    <!-- Search Bar -->
    <div class="relative mb-6">
      <input type="text" id="search-input" placeholder="Search reviews..." class="w-full px-4
py-3 pl-10 rounded-lg border border-gray-300 focus:ring-2 focus:ring-[#0088cc]
focus:border-transparent outline-none">
      <svg class="absolute left-3 top-3.5 w-5 h-5 text-gray-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M21
21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"></path>
      </svg>
    </div>

    <!-- Reviews Container -->
    <div id="reviews-loading" class="flex justify-center py-12 hidden">
      <div class="loading-spinner"></div>
    </div>

    <div id="reviews-container" class="reviews-container space-y-6 pr-2">
      <!-- Reviews will be loaded here dynamically -->
      <div class="text-center py-12 text-gray-500">
        Loading reviews...
      </div>
    </div>

    <!-- Pagination -->
```

```
    <div id="pagination" class="flex justify-center gap-2 mt-8">
      <!-- Pagination will be loaded here -->
    </div>
   </div>
  </section>

  <!-- Write Review Modal -->
  <div id="review-modal" class="fixed inset-0 bg-black/50 flex items-center justify-center
z-50 hidden">
   <div class="bg-white rounded-2xl p-8 max-w-lg w-full mx-4 transform transition-all
max-h-[90vh] overflow-y-auto">
    <div class="flex justify-between items-center mb-6">
     <h3 class="text-xl font-bold text-[#1a365d]">Write a Review</h3>
     <button id="close-review-modal" class="text-gray-400 hover:text-gray-600
text-2xl">×</button>
    </div>

    <form id="review-form" class="space-y-4">
     <!-- Rating -->
     <div>
      <label class="block text-sm font-medium text-gray-700 mb-2">Your Rating *</label>
      <div class="flex gap-2 text-3xl" id="rating-stars">
       <span class="star cursor-pointer hover:scale-110 transition" data-rating="1">☆</span>
       <span class="star cursor-pointer hover:scale-110 transition" data-rating="2">☆</span>
       <span class="star cursor-pointer hover:scale-110 transition" data-rating="3">☆</span>
       <span class="star cursor-pointer hover:scale-110 transition" data-rating="4">☆</span>
       <span class="star cursor-pointer hover:scale-110 transition" data-rating="5">☆</span>
      </div>
      <input type="hidden" id="review-rating" name="rating" required>
     </div>

     <!-- Title -->
     <div>
      <label for="review-title" class="block text-sm font-medium text-gray-700 mb-1">Review
Title</label>
      <input type="text" id="review-title" class="w-full px-4 py-2.5 border border-gray-300
rounded-lg focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none"
placeholder="Summarize your experience">
     </div>

     <!-- Content -->
     <div>
      <label for="review-content" class="block text-sm font-medium text-gray-700 mb-1">Your
Review *</label>
      <textarea id="review-content" rows="4" class="w-full px-4 py-2.5 border border-gray-300
rounded-lg focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none
resize-none" placeholder="Tell us about your experience..." required></textarea>
      <p class="text-xs text-gray-500 mt-1">Minimum 10 characters</p>
```

```
      </div>

      <!-- Hospital (Optional) -->
      <div>
       <label for="review-hospital" class="block text-sm font-medium text-gray-700
mb-1">Hospital (Optional)</label>
       <input type="text" id="review-hospital" class="w-full px-4 py-2.5 border border-gray-300
rounded-lg focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none"
placeholder="e.g., City General Hospital">
      </div>

      <!-- Doctor (Optional) -->
      <div>
       <label for="review-doctor" class="block text-sm font-medium text-gray-700
mb-1">Doctor (Optional)</label>
       <input type="text" id="review-doctor" class="w-full px-4 py-2.5 border border-gray-300
rounded-lg focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none"
placeholder="e.g., Dr. Sarah Johnson">
      </div>

      <div class="text-xs text-gray-500 bg-blue-50 p-3 rounded-lg">
        <span class="font-semibold">Note:</span> Your review will be public and help others
make informed decisions.
      </div>

      <button type="submit" class="w-full bg-gradient-to-r from-[#0088cc] to-[#00aadd]
hover:from-[#0077bb] hover:to-[#0099cc] text-white py-3 rounded-lg font-semibold
transition-all hover:shadow-lg">
       Submit Review
      </button>
     </form>

     <div id="review-success" class="hidden text-center py-8">
      <div class="text-5xl mb-4">✅</div>
      <h4 class="text-xl font-bold text-[#1a365d] mb-2">Thank You!</h4>
      <p class="text-gray-600">Your review has been submitted and will be visible shortly.</p>
     </div>
    </div>
   </div>

   <!-- Footer -->
   <footer class="bg-gradient-to-br from-[#006d77] to-[#004d55] text-white py-16 relative
overflow-hidden">
    <div class="floating-shape top-10 right-20 w-64 h-64 bg-[#0088cc]"
style="animation-delay: 0s; opacity: 0.1;"></div>
    <div class="floating-shape bottom-10 left-20 w-80 h-80 bg-[#00aadd]"
style="animation-delay: 3s; opacity: 0.1;"></div>
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 relative z-10">
```

```
<div class="grid md:grid-cols-4 gap-10">
 <div class="md:col-span-2">
  <div class="flex items-center gap-2 mb-4">
   <svg class="w-10 h-10" viewbox="0 0 50 50" fill="none">
    <circle cx="25" cy="25" r="23" fill="#0088cc" opacity="0.2" />
    <path d="M25 8C15.6 8 8 15.6 8 25s7.6 17 17 17 17-7.6 17-17S34.4 8 25 8z"
stroke="#00aadd" stroke-width="2" fill="none" />
    <path d="M25 15v20M15 25h20" stroke="#00aadd" stroke-width="3"
stroke-linecap="round" />
   </svg>
   <span class="text-2xl font-bold">BondHealth <span
class="text-[#00aadd]">Connect</span></span>
  </div>
  <p class="text-gray-400 mb-6 max-w-md">Unifying healthcare across the state. Your
health journey, simplified and secured.</p>
  <div class="space-y-2">
   <p class="flex items-center gap-3"><span class="text-[#00aadd]">📧</span> <a
id="support-email" href="mailto:support@bondhealth.com" class="hover:text-[#00aadd]
transition-colors">support@bondhealth.com</a></p>
   <p class="flex items-center gap-3"><span class="text-[#00aadd]">📞</span> <span
id="helpline">1-800-BOND-HLTH</span> <span class="text-xs bg-[#00aadd] text-white px-2
py-0.5 rounded-full">24/7</span></p>
  </div>
 </div>
 <div>
  <h4 class="font-semibold text-lg mb-4">Quick Links</h4>
  <ul class="space-y-3">
   <li><a href="#" class="text-cyan-100 hover:text-[#00aadd] transition-all
hover:translate-x-1 inline-block">Patient Privacy Rights</a></li>
   <li><a href="#" class="text-cyan-100 hover:text-[#00aadd] transition-all
hover:translate-x-1 inline-block">Terms of Service</a></li>
   <li><a href="#" class="text-cyan-100 hover:text-[#00aadd] transition-all
hover:translate-x-1 inline-block">Help Center</a></li>
  </ul>
 </div>
 <div>
  <h4 class="font-semibold text-lg mb-4">Stay Connected</h4>
  <div class="flex gap-3">
   <a href="#" class="w-10 h-10 bg-white/10 hover:bg-gradient-to-br hover:from-[#0088cc]
hover:to-[#00aadd] rounded-full flex items-center justify-center transition-all hover:scale-110
hover:shadow-lg hover:shadow-blue-300">
    <svg class="w-5 h-5" fill="currentColor" viewbox="0 0 24 24"><path d="M24
4.557c-.883.392-1.832.656-2.828.775 1.017-.609 1.798-1.574
2.165-2.724-.951.564-2.005.974-3.127 1.195-.897-.957-2.178-1.555-3.594-1.555-3.179
0-5.515 2.966-4.797 6.045-4.091-.205-7.719-2.165-10.148-5.144-1.29 2.213-.669 5.108
1.523 6.574-.806-.026-1.566-.247-2.229-.616-.054 2.281 1.581 4.415 3.949
4.89-.693.188-1.452.232-2.224.084.626 1.956 2.444 3.379 4.6 3.419-2.07 1.623-4.678
```

2.348-7.29 2.04 2.179 1.397 4.768 2.212 7.548 2.212 9.142 0 14.307-7.721 13.995-14.646.962-.695 1.797-1.562 2.457-2.549z" /></svg>
</a>
<a href="#" class="w-10 h-10 bg-white/10 hover:bg-gradient-to-br hover:from-[#0088cc] hover:to-[#00aadd] rounded-full flex items-center justify-center transition-all hover:scale-110 hover:shadow-lg hover:shadow-blue-300">
<svg class="w-5 h-5" fill="currentColor" viewbox="0 0 24 24"><path d="M12 2.163c3.204 0 3.584.012 4.85.07 3.252.148 4.771 1.691 4.919 4.919.058 1.265.069 1.645.069 4.849 0 3.205-.012 3.584-.069 4.849-.149 3.225-1.664 4.771-4.919 4.919-1.266.058-1.644.07-4.85.07-3.204 0-3.584-.012-4.849-.07-3.26-.149-4.771-1.699-4.919-4.92-.058-1.265-.07-1.644-.07-4.849 0-3.204.013-3.583.07-4.849.149-3.227 1.664-4.771 4.919-4.919 1.266-.057 1.645-.069 4.849-.069zm0-2.163c-3.259 0-3.667.014-4.947.072-4.358.2-6.78 2.618-6.98 6.98-.059 1.281-.073 1.689-.073 4.948 0 3.259.014 3.668.072 4.948.2 4.358 2.618 6.78 6.98 6.98 1.281.058 1.689.072 4.948.072 3.259 0 3.668-.014 4.948-.072 4.354-.2 6.782-2.618 6.979-6.98.059-1.28.073-1.689.073-4.948 0-3.259-.014-3.667-.072-4.947-.196-4.354-2.617-6.78-6.979-6.98-1.281-.059-1.69-.073-4.949-.073zm0 5.838c-3.403 0-6.162 2.759-6.162 6.162s2.759 6.163 6.162 6.163 6.162-2.759 6.162-6.163c0-3.403-2.759-6.162-6.162-6.162zm0 10.162c-2.209 0-4-1.79-4-4 0-2.209 1.791-4 4-4s4 1.791 4 4c0 2.21-1.791 4-4 4zm6.406-11.845c-.796 0-1.441.645-1.441 1.44s.645 1.44 1.441 1.44c.795 0 1.439-.645 1.439-1.44s-.644-1.44-1.439-1.44z" /></svg>
</a>
<a href="#" class="w-10 h-10 bg-white/10 hover:bg-gradient-to-br hover:from-[#0088cc] hover:to-[#00aadd] rounded-full flex items-center justify-center transition-all hover:scale-110 hover:shadow-lg hover:shadow-blue-300">
<svg class="w-5 h-5" fill="currentColor" viewbox="0 0 24 24"><path d="M20.447 20.452h-3.554v-5.569c0-1.328-.027-3.037-1.852-3.037-1.853 0-2.136 1.445-2.136 2.939v5.667H9.351V9h3.414v1.561h.046c.477-.9 1.637-1.85 3.37-1.85 3.601 0 4.267 2.37 4.267 5.455v6.286zM5.337 7.433c-1.144 0-2.063-.926-2.063-2.065 0-1.138.92-2.063 2.063-2.063 1.14 0 2.064.925 2.064 2.063 0 1.139-.925 2.065-2.064 2.065zm1.782 13.019H3.555V9h3.564v11.452zM22.225 0H1.771C.792 0 0 .774 0 1.729v20.542C0 23.227.792 24 1.771 24h20.451C23.2 24 24 23.227 24 22.271V1.729C24 .774 23.2 0 22.222 0h.003z" /></svg>
</a>

© 2024 BondHealth. All rights reserved. | HIPAA Compliant | SOC 2 Certified

</footer>

<!-- Floating Feedback Button -->
<button id="feedback-btn" class="fixed bottom-6 right-6 w-14 h-14 bg-gradient-to-br from-[#0088cc] to-[#00aadd] hover:from-[#0077bb] hover:to-[#0099cc] text-white rounded-full shadow-lg hover:shadow-xl transition-all flex items-center justify-center text-2xl animate-pulse-glow hover:scale-110 z-50">💬 </button>

```
  <!-- Feedback Modal -->
  <div id="feedback-modal" class="fixed inset-0 bg-black/50 flex items-center justify-center z-50 hidden">
    <div class="bg-white rounded-2xl p-8 max-w-md w-full mx-4 transform transition-all">
      <div class="flex justify-between items-center mb-6">
        <h3 class="text-xl font-bold text-[#1a365d]">Send Feedback</h3>
        <button id="close-modal" class="text-gray-400 hover:text-gray-600 text-2xl">×</button>
      </div>
      <form id="feedback-form" class="space-y-4">
        <div><label for="feedback-name" class="block text-sm font-medium text-gray-700 mb-1">Name</label> <input type="text" id="feedback-name" class="w-full px-4 py-2.5 border border-gray-300 rounded-lg focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none" placeholder="Your name"></div>
        <div><label for="feedback-email" class="block text-sm font-medium text-gray-700 mb-1">Email</label> <input type="email" id="feedback-email" class="w-full px-4 py-2.5 border border-gray-300 rounded-lg focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none" placeholder="your@email.com"></div>
        <div><label for="feedback-message" class="block text-sm font-medium text-gray-700 mb-1">Message</label> <textarea id="feedback-message" rows="4" class="w-full px-4 py-2.5 border border-gray-300 rounded-lg focus:ring-2 focus:ring-[#0088cc] focus:border-transparent outline-none resize-none" placeholder="Tell us what you think..."></textarea></div>
        <button type="submit" class="w-full bg-gradient-to-r from-[#0088cc] to-[#00aadd] hover:from-[#0077bb] hover:to-[#0099cc] text-white py-3 rounded-lg font-semibold transition-all hover:shadow-lg hover:scale-105"> Send Feedback </button>
      </form>
      <div id="feedback-success" class="hidden text-center py-8">
        <div class="text-5xl mb-4">✅</div>
        <h4 class="text-xl font-bold text-[#1a365d] mb-2">Thank You!</h4>
        <p class="text-gray-600">Your feedback has been received.</p>
      </div>
    </div>
  </div>
</div>

<script>
  const defaultConfig = {
    tagline: "Unifying Care, Strengthening Lives.",
    about_title: "About BondHealth",
    about_text: "BondHealth is a state-wide digital ecosystem that bridges the gap between you and your medical providers. We securely centralize your health records, ensuring that your medical history, appointments, and lab results are accessible at every hospital you visit.",
    support_email: "support@bondhealth.com",
    helpline: "1-800-BOND-HLTH",
    primary_color: "#0088cc",
    secondary_color: "#00aadd",
```

```
    background_color: "#ffffff",
    text_color: "#006d77",
    surface_color: "#e0ffff",
    font_family: "Poppins",
    font_size: 16
  };

  let config = { ...defaultConfig };

  async function onConfigChange(newConfig) {
    config = { ...config, ...newConfig };

    const taglineEl = document.getElementById('tagline');
    if (taglineEl) {
      const taglineText = config.tagline || defaultConfig.tagline;
      const parts = taglineText.split(',');
      if (parts.length > 1) {
        taglineEl.innerHTML = \`\${parts[0]},<br><span
class="gradient-text">\${parts.slice(1).join(',').trim()}</span>\`;
      } else {
        taglineEl.textContent = taglineText;
      }
    }

    const aboutTextEl = document.getElementById('about-text');
    if (aboutTextEl) {
      aboutTextEl.textContent = config.about_text || defaultConfig.about_text;
    }

    const supportEmailEl = document.getElementById('support-email');
    if (supportEmailEl) {
      const email = config.support_email || defaultConfig.support_email;
      supportEmailEl.href = \`mailto:\${email}\`;
      supportEmailEl.textContent = email;
    }

    const helplineEl = document.getElementById('helpline');
    if (helplineEl) {
      helplineEl.textContent = config.helpline || defaultConfig.helpline;
    }

    const fontFamily = config.font_family || defaultConfig.font_family;
    document.body.style.fontFamily = \`\${fontFamily}, sans-serif\`;

    const baseSize = config.font_size || defaultConfig.font_size;
    document.documentElement.style.fontSize = \`\${baseSize}px\`;
  }
```

```
function mapToCapabilities(config) {
  return {
    recolorables: [
      {
        get: () => config.background_color || defaultConfig.background_color,
        set: (value) => {
          config.background_color = value;
          if (window.elementSdk) window.elementSdk.setConfig({ background_color: value });
        }
      },
      {
        get: () => config.surface_color || defaultConfig.surface_color,
        set: (value) => {
          config.surface_color = value;
          if (window.elementSdk) window.elementSdk.setConfig({ surface_color: value });
        }
      },
      {
        get: () => config.text_color || defaultConfig.text_color,
        set: (value) => {
          config.text_color = value;
          if (window.elementSdk) window.elementSdk.setConfig({ text_color: value });
        }
      },
      {
        get: () => config.primary_color || defaultConfig.primary_color,
        set: (value) => {
          config.primary_color = value;
          if (window.elementSdk) window.elementSdk.setConfig({ primary_color: value });
        }
      },
      {
        get: () => config.secondary_color || defaultConfig.secondary_color,
        set: (value) => {
          config.secondary_color = value;
          if (window.elementSdk) window.elementSdk.setConfig({ secondary_color: value });
        }
      }
    ],
    borderables: [],
    fontEditable: {
      get: () => config.font_family || defaultConfig.font_family,
      set: (value) => {
        config.font_family = value;
        if (window.elementSdk) window.elementSdk.setConfig({ font_family: value });
      }
    },
    fontSizeable: {
```

```
      get: () => config.font_size || defaultConfig.font_size,
      set: (value) => {
        config.font_size = value;
        if (window.elementSdk) window.elementSdk.setConfig({ font_size: value });
      }
    }
  };
}

function mapToEditPanelValues(config) {
  return new Map([
    ["tagline", config.tagline || defaultConfig.tagline],
    ["about_title", config.about_title || defaultConfig.about_title],
    ["about_text", config.about_text || defaultConfig.about_text],
    ["support_email", config.support_email || defaultConfig.support_email],
    ["helpline", config.helpline || defaultConfig.helpline]
  ]);
}

if (window.elementSdk) {
  window.elementSdk.init({
    defaultConfig,
    onConfigChange,
    mapToCapabilities,
    mapToEditPanelValues
  });
}

// Review System JavaScript
let currentFilter = 'all';
let currentSort = 'recent';
let currentSearch = '';
let currentPage = 1;

// Load reviews on page load
document.addEventListener('DOMContentLoaded', function() {
  loadReviews();

  // Setup filter buttons
  document.querySelectorAll('.filter-btn').forEach(btn => {
    btn.addEventListener('click', function() {
      document.querySelectorAll('.filter-btn').forEach(b => b.classList.remove('active'));
      this.classList.add('active');
      currentFilter = this.dataset.filter;
      currentPage = 1;
      loadReviews();
    });
  });
```

```
  // Setup sort select
  document.getElementById('sort-select').addEventListener('change', function() {
    currentSort = this.value;
    currentPage = 1;
    loadReviews();
  });

  // Setup search with debounce
  let searchTimeout;
  document.getElementById('search-input').addEventListener('input', function() {
    clearTimeout(searchTimeout);
    searchTimeout = setTimeout(() => {
      currentSearch = this.value;
      currentPage = 1;
      loadReviews();
    }, 500);
  });

  // Setup review modal
  setupReviewModal();
});

function loadReviews() {
  const loadingEl = document.getElementById('reviews-loading');
  const reviewsContainer = document.getElementById('reviews-container');

  loadingEl.classList.remove('hidden');
  reviewsContainer.innerHTML = '';

  let url = \`/api/reviews?sort=\${currentSort}&page=\${currentPage}&limit=10\`;

  if (currentFilter !== 'all') {
    url += \`&rating=\${currentFilter}\`;
  }

  if (currentSearch) {
    url += \`&search=\${encodeURIComponent(currentSearch)}\`;
  }

  fetch(url)
    .then(res => res.json())
    .then(data => {
      loadingEl.classList.add('hidden');

      if (data.success) {
        displayReviews(data.reviews);
        updateRatingStats(data.stats);
```

```
          updatePagination(data.pagination);
        } else {
          reviewsContainer.innerHTML = '<div class="text-center py-12 text-gray-500">Failed
to load reviews.</div>';
        }
      })
      .catch(err => {
        console.error('Error loading reviews:', err);
        loadingEl.classList.add('hidden');
        reviewsContainer.innerHTML = '<div class="text-center py-12 text-gray-500">Error
loading reviews.</div>';
      });
  }

  function displayReviews(reviews) {
    const container = document.getElementById('reviews-container');

    if (!reviews || reviews.length === 0) {
      container.innerHTML = '<div class="text-center py-12 text-gray-500">No reviews found.
Be the first to write a review!</div>';
      return;
    }

    let html = '';

    reviews.forEach(review => {
      const initials = review.reviewer_name.split(' ').map(n => n[0]).join('').substring(0,
2).toUpperCase();
      const stars = '★'.repeat(review.rating) + '☆'.repeat(5 - review.rating);
      const verifiedBadge = review.is_verified ? '<span class="ml-2 text-xs bg-green-100
text-green-700 px-2 py-0.5 rounded-full">Verified Patient</span>' : '';

      html += \`
        <div class="review-bubble bg-white p-6 rounded-2xl shadow-lg border
border-gray-100">
          <div class="flex items-start gap-4">
            <div class="w-12 h-12 rounded-full bg-gradient-to-br from-[#0056b3] to-[#0088cc]
flex items-center justify-center text-white font-bold text-lg flex-shrink-0">\${initials}</div>
            <div class="flex-1">
              <div class="flex flex-wrap items-center gap-2 mb-2">
                <span class="font-semibold text-[#1a365d]">\${review.reviewer_name}</span>
                \${verifiedBadge}
                <div class="flex text-yellow-400 text-sm ml-auto">\${stars}</div>
              </div>
              \${review.title ? '<h4 class="font-semibold text-gray-800 mb-2">' + review.title +
'</h4>' : ''}
              <p class="text-gray-600 text-sm leading-relaxed">"\${review.content}"</p>
              <div class="flex flex-wrap items-center justify-between mt-3">
```

```
                <div class="flex items-center gap-4">
                  <span class="text-xs text-gray-400">\${review.formatted_date}</span>
                  \${review.hospital_name ? '<span class="text-xs bg-blue-50 text-blue-600 px-2
py-0.5 rounded-full">' + review.hospital_name + '</span>' : ''}
                  \${review.doctor_name ? '<span class="text-xs bg-green-50 text-green-600 px-2
py-0.5 rounded-full">' + review.doctor_name + '</span>' : ''}
                </div>
                <button class="helpful-btn text-xs text-gray-500 hover:text-[#0088cc]
transition-colors" data-id="\${review.review_id}">
                  👍 Helpful (\${review.helpful_count})
                </button>
              </div>
            </div>
          </div>
        </div>
      \`;
    });

    container.innerHTML = html;

    // Add helpful button listeners
    document.querySelectorAll('.helpful-btn').forEach(btn => {
      btn.addEventListener('click', function() {
        const reviewId = this.dataset.id;
        fetch(\`/api/reviews/\${reviewId}/helpful\`, { method: 'POST' })
          .then(res => res.json())
          .then(data => {
            if (data.success) {
              this.innerHTML = \`👍 Helpful (\${data.helpful_count})\`;
            }
          });
      });
    });
  }

  function updateRatingStats(stats) {
    if (!stats) return;

    document.getElementById('average-rating').textContent =
parseFloat(stats.average_rating).toFixed(1);
    document.getElementById('total-reviews').textContent = \`Based on
\${stats.total_reviews} reviews\`;

    const total = parseInt(stats.total_reviews) || 1;
    document.getElementById('five-star-bar').style.width = (stats.five_star / total * 100) + '%';
    document.getElementById('four-star-bar').style.width = (stats.four_star / total * 100) +
'%';
```

```
    document.getElementById('three-star-bar').style.width = (stats.three_star / total * 100) +
'%';
    document.getElementById('two-star-bar').style.width = (stats.two_star / total * 100) + '%';
    document.getElementById('one-star-bar').style.width = (stats.one_star / total * 100) + '%';

    document.getElementById('five-star-count').textContent = stats.five_star;
    document.getElementById('four-star-count').textContent = stats.four_star;
    document.getElementById('three-star-count').textContent = stats.three_star;
    document.getElementById('two-star-count').textContent = stats.two_star;
    document.getElementById('one-star-count').textContent = stats.one_star;
  }

  function updatePagination(pagination) {
    const paginationEl = document.getElementById('pagination');

    if (!pagination || pagination.totalPages <= 1) {
      paginationEl.innerHTML = '';
      return;
    }

    let html = '';

    // Previous button
    html += \`
      <button class="px-4 py-2 rounded-lg border \${pagination.currentPage === 1 ?
'text-gray-400 border-gray-200 cursor-not-allowed' : 'text-[#0088cc] border-[#0088cc]
hover:bg-[#0088cc] hover:text-white'} transition-all"
        \${pagination.currentPage === 1 ? 'disabled' : 'onclick="goToPage(' +
(pagination.currentPage - 1) + ')'"}>
        Previous
      </button>
    \`;

    // Page numbers
    for (let i = 1; i <= pagination.totalPages; i++) {
      if (i === pagination.currentPage ||
          i === 1 ||
          i === pagination.totalPages ||
          Math.abs(i - pagination.currentPage) <= 2) {
        html += \`
          <button class="px-4 py-2 rounded-lg border \${i === pagination.currentPage ?
'bg-[#0088cc] text-white border-[#0088cc]' : 'text-gray-600 border-gray-300
hover:bg-gray-100'} transition-all"
            onclick="goToPage(\${i})">
            \${i}
          </button>
        \`;
      } else if (i === pagination.currentPage - 3 || i === pagination.currentPage + 3) {
```

```
        html += '<span class="px-2">...</span>';
      }
    }

    // Next button
    html += \`
      <button class="px-4 py-2 rounded-lg border \${pagination.currentPage ===
pagination.totalPages ? 'text-gray-400 border-gray-200 cursor-not-allowed' : 'text-[#0088cc]
border-[#0088cc] hover:bg-[#0088cc] hover:text-white'} transition-all"
        \${pagination.currentPage === pagination.totalPages ? 'disabled' :
'onclick="goToPage(' + (pagination.currentPage + 1) + ')"'}>
        Next
      </button>
    \`;

    paginationEl.innerHTML = html;
  }

  window.goToPage = function(page) {
    currentPage = page;
    loadReviews();
    document.getElementById('reviews').scrollIntoView({ behavior: 'smooth' });
  };

  function setupReviewModal() {
    const modal = document.getElementById('review-modal');
    const openBtn = document.getElementById('write-review-btn');
    const closeBtn = document.getElementById('close-review-modal');
    const stars = document.querySelectorAll('#rating-stars .star');
    const ratingInput = document.getElementById('review-rating');
    const form = document.getElementById('review-form');
    const successEl = document.getElementById('review-success');

    let selectedRating = 0;

    openBtn.addEventListener('click', () => {
      // Check if user is logged in
      fetch('/api/auth/me')
        .then(res => res.json())
        .then(data => {
          if (data.success) {
            modal.classList.remove('hidden');
          } else {
            alert('Please sign in to write a review');
            window.location.href = '/signin';
          }
        })
        .catch(() => {
```

```
      alert('Please sign in to write a review');
      window.location.href = '/signin';
    });
});

closeBtn.addEventListener('click', () => {
  modal.classList.add('hidden');
  resetReviewForm();
});

modal.addEventListener('click', (e) => {
  if (e.target === modal) {
    modal.classList.add('hidden');
    resetReviewForm();
  }
});

// Rating stars
stars.forEach(star => {
  star.addEventListener('mouseover', function() {
    const rating = parseInt(this.dataset.rating);
    stars.forEach((s, i) => {
      s.textContent = i < rating ? '★' : '☆';
    });
  });

  star.addEventListener('mouseout', function() {
    stars.forEach((s, i) => {
      s.textContent = i < selectedRating ? '★' : '☆';
    });
  });

  star.addEventListener('click', function() {
    selectedRating = parseInt(this.dataset.rating);
    ratingInput.value = selectedRating;
    stars.forEach((s, i) => {
      s.textContent = i < selectedRating ? '★' : '☆';
    });
  });
});

form.addEventListener('submit', async (e) => {
  e.preventDefault();

  if (!selectedRating) {
    alert('Please select a rating');
    return;
  }
```

```
    const content = document.getElementById('review-content').value;
    if (content.length < 10) {
      alert('Review content must be at least 10 characters');
      return;
    }

    const formData = {
      rating: selectedRating,
      title: document.getElementById('review-title').value,
      content: content,
      hospital_name: document.getElementById('review-hospital').value || null,
      doctor_name: document.getElementById('review-doctor').value || null
    };

    try {
      const response = await fetch('/api/reviews', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData)
      });

      const data = await response.json();

      if (data.success) {
        form.classList.add('hidden');
        successEl.classList.remove('hidden');
        setTimeout(() => {
          modal.classList.add('hidden');
          resetReviewForm();
          loadReviews(); // Reload reviews to show the new one
        }, 2000);
      } else {
        alert(data.message || 'Failed to submit review');
      }
    } catch (error) {
      console.error('Error submitting review:', error);
      alert('Network error. Please try again.');
    }
  });
}

function resetReviewForm() {
  const form = document.getElementById('review-form');
  const successEl = document.getElementById('review-success');
  const stars = document.querySelectorAll('#rating-stars .star');

  form.classList.remove('hidden');
```

```
  successEl.classList.add('hidden');
  form.reset();

  stars.forEach(s => s.textContent = '☆');
  document.getElementById('review-rating').value = '';
}

// Feedback modal functionality
const feedbackBtn = document.getElementById('feedback-btn');
const feedbackModal = document.getElementById('feedback-modal');
const closeModal = document.getElementById('close-modal');
const feedbackForm = document.getElementById('feedback-form');
const feedbackSuccess = document.getElementById('feedback-success');

feedbackBtn.addEventListener('click', () => {
  feedbackModal.classList.remove('hidden');
});

closeModal.addEventListener('click', () => {
  feedbackModal.classList.add('hidden');
  feedbackForm.classList.remove('hidden');
  feedbackSuccess.classList.add('hidden');
  feedbackForm.reset();
});

feedbackModal.addEventListener('click', (e) => {
  if (e.target === feedbackModal) {
    feedbackModal.classList.add('hidden');
    feedbackForm.classList.remove('hidden');
    feedbackSuccess.classList.add('hidden');
    feedbackForm.reset();
  }
});

feedbackForm.addEventListener('submit', async (e) => {
  e.preventDefault();

  const formData = {
    name: document.getElementById('feedback-name').value,
    email: document.getElementById('feedback-email').value,
    message: document.getElementById('feedback-message').value
  };

  try {
    const response = await fetch('/api/feedback', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(formData)
```

```
      });

      if (response.ok) {
        feedbackForm.classList.add('hidden');
        feedbackSuccess.classList.remove('hidden');
        setTimeout(() => {
          feedbackModal.classList.add('hidden');
          feedbackForm.classList.remove('hidden');
          feedbackSuccess.classList.add('hidden');
          feedbackForm.reset();
        }, 2000);
      } else {
        alert('Failed to submit feedback. Please try again.');
      }
    } catch (error) {
      console.error('Error submitting feedback:', error);
      alert('Network error. Please try again.');
    }
  });

  // Smooth scroll for navigation
  document.querySelectorAll('a[href^="#"]').forEach(anchor => {
    anchor.addEventListener('click', function (e) {
      e.preventDefault();
      const target = document.querySelector(this.getAttribute('href'));
      if (target) {
        target.scrollIntoView({ behavior: 'smooth' });
      }
    });
  });

  onConfigChange(config);
 </script>
</body>
</html>`;
}


// ==============================================
// DASHBOARD PROTECTION MIDDLEWARE
// ==============================================


// ==============================================
// DASHBOARD ROUTES (Copied from signin.js)
// ==============================================

// SERVE LAB DASHBOARD - Simple extraction, no data needed
```

```
// SERVE LAB DASHBOARD
app.get('/lab-dashboard', requireAuth('lab'), async (req, res) => {
    try {
        const renderLabDashboard = require('./labs.js');
        // Pass the actual user ID from the authenticated user
        const html = await renderLabDashboard(req.user.id);
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading labs.js:', err);
        res.status(500).send(`
            <h1>500 - Lab Dashboard not found</h1>
            <p>Make sure labs.js is in the same directory and exports the render function</p>
            <a href="/signin">← Back to Sign In</a>
        `);
    }
});


// SERVE PATIENT DASHBOARD - Auto-extract everything from Patient.js
// SERVE PATIENT DASHBOARD
app.get('/patient-dashboard', requireAuth('patient'), async(req, res) => {
    try {
        const renderPatientDashboard = require('./Patient.js');
        const html = await renderPatientDashboard(req.user.id); // Pass user ID
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading Patient.js:', err);
        res.status(500).send(`
            <h1>500 - Patient Dashboard not found</h1>
            <p>Make sure Patient.js is in the same directory and exports the render function</p>
            <a href="/signin">← Back to Sign In</a>
        `);
    }
});

app.get('/admin-dashboard', requireAuth('admin'), async (req, res) => {
    try {
        const renderAdminDashboard = require('./admin.js');
        const html = await renderAdminDashboard(req.user.id); // Pass user ID
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading admin.js:', err);
        res.status(500).send(`
            <h1>500 - Admin Dashboard not found</h1>
            <p>Make sure admin.js is in the same directory and exports the render function</p>
```

```
            <a href="/signin">← Back to Sign In</a>
        `);
    }
});


// SERVE DOCTOR DASHBOARD - USING EXPORTED FUNCTION FROM Doctor.js
// SERVE DOCTOR DASHBOARD
app.get('/doctor-dashboard', requireAuth('doctor'), async(req, res) => {
    try {
        const renderDoctorDashboard = require('./Doctor.js');
        const html = await renderDoctorDashboard(req.user.id); // Pass user ID
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading Doctor.js:', err);
        res.status(500).send(`
            <h1>500 - Doctor Dashboard not found</h1>
            <p>Make sure Doctor.js is in the same directory and exports the render function</p>
            <a href="/signin">← Back to Sign In</a>
        `);
    }
});

// SERVE ADMIN SIGNUP
// SERVE HOSPITAL/ADMIN REGISTRATION
// SERVE ADMIN/HOSPITAL REGISTRATION - Complete version from
HospitalRegistration.js
// SERVE HOSPITAL/ADMIN REGISTRATION - Using external file
app.get('/admin-signup', (req, res) => {
    try {
        const renderHospitalRegistration = require('./HospitalRegistration.js');
        const html = renderHospitalRegistration();
        res.setHeader('Content-Type', 'text/html');
        res.send(html);
    } catch (err) {
        console.error('Error loading HospitalRegistration.js:', err);
        res.status(500).send(`
            <h1>500 - Hospital Registration not found</h1>
            <p>Make sure HospitalRegistration.js is in the same directory.</p>
            <a href="/signin">← Back to Sign In</a>
        `);
    }
});

// Start the server
app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
```

```
    console.log(`Sign In page: http://localhost:${PORT}/signin.js`);
    console.log(`Press Ctrl+C to stop the server`);
});
```